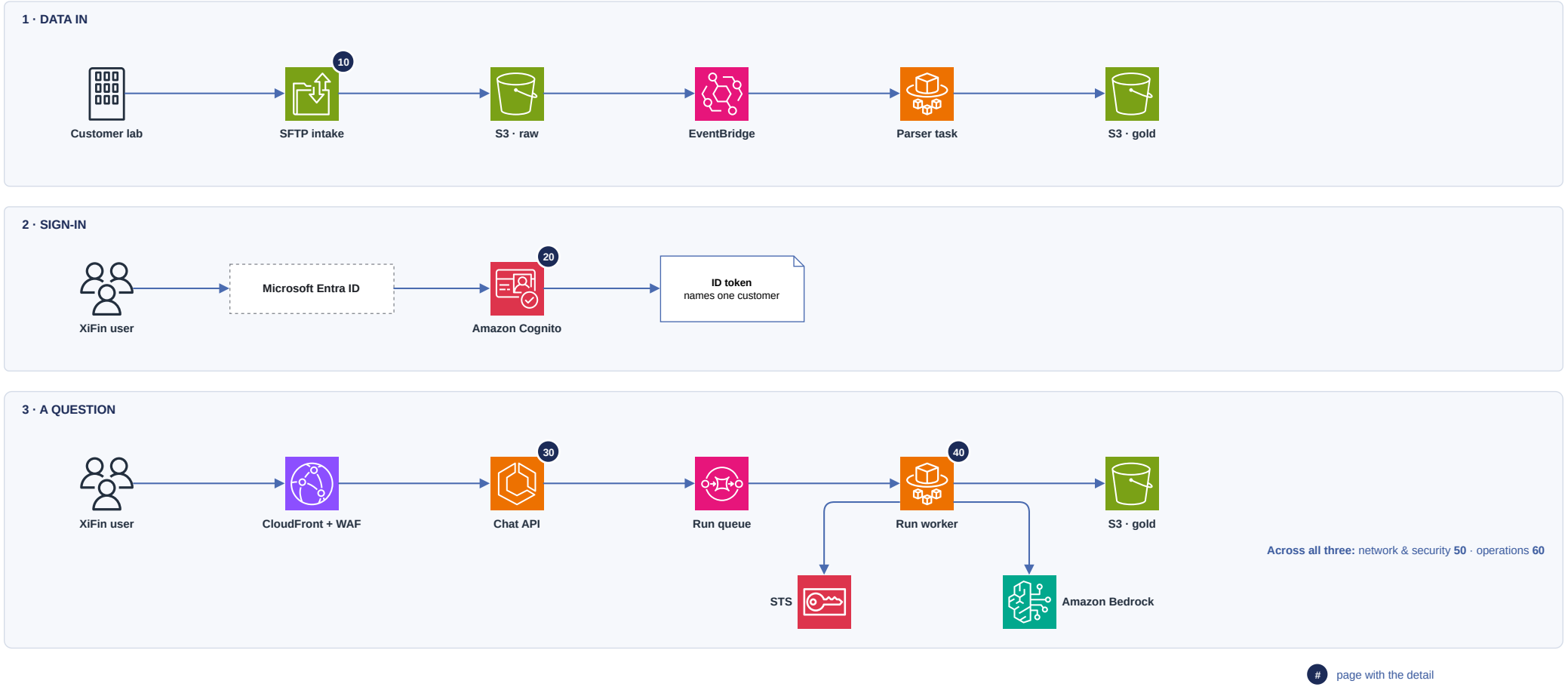
ClaimClarity architecture overview

ClaimClarity is a chat assistant for XiFin staff. It answers questions about a clinical lab customer's claims (837P) and remittances (835), and shows the SQL and data behind every answer. Each customer's data stays inside that customer's own S3 prefix, KMS keys and IAM roles. A signed-in user reaches only the customer their token names.

CLAIMCLARITY · HOW IT WORKS



In this diagram

Item	What it does	Details
Data in		Ingestion & lake
Customer lab	Sends daily 837, 835 and 277 files	SFTP intake
SFTP intake	AWS Transfer Family receives the files for that customer	SFTP intake
S3 · raw	The files exactly as received, write-once	S3 layout
EventBridge	Starts a parser task when a batch is complete	Parse trigger
Parser task	Fargate task, running as the customer's parser role. Builds the lossless and gold tables.	Parser task
S3 · gold	The curated tables the chat reads	Gold publish
Sign-in		Identity & access
Microsoft Entra ID	Company sign-in	Sign-in
Amazon Cognito	Issues the ID token that names one customer	Sign-in
A question		Chat backend
CloudFront + WAF	The only public entry: serves the app and passes API calls on	Edge
Chat API	Checks the token and queues one run per question	One chat turn
Run queue	SQS queue between the chat API and the run workers	Limits
Run worker	Fargate task that runs the analyzer for one run	Analyzer
STS	Exchanges the token for 15-minute credentials of the customer's reader role	Token to credentials
Amazon Bedrock	Claude Haiku: every model call	Model calls

Applying to all three flows: Network & security · Operations

How it works

1. **Data in.** A lab's files land in its raw prefix. When the batch is complete, a parser task, running as that customer's role, writes the lossless tables and publishes a new gold version.
2. **Sign-in.** The user signs in with Entra ID. Cognito issues an ID token naming the customer, which the browser keeps in an httpOnly session cookie.
3. **A question.**
    - The chat API checks the token and queues a run.
    - A run worker gets the customer's reader credentials from STS.
    - The analyzer answers from the current gold version: with a preset KPI query when one fits, otherwise with its own read-only SQL.
    - The server re-runs the final SQL, so the numbers come from the database.

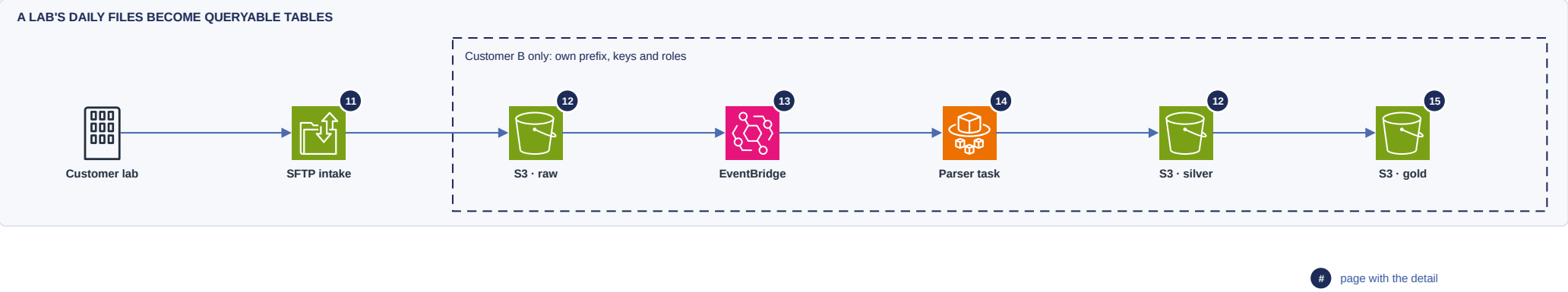
Appendix: AWS service catalog · IAM and KMS matrix · Glossary · Open decisions

The examples use a synthetic data set with no real patient data.

Ingestion & lake

A customer lab's daily 837P, 835 and 277 files arrive over SFTP and are kept, unchanged and locked, in that customer's raw zone. One short-lived parser task per batch turns them into lossless tables (silver) and publishes a new version of the semantic tables (gold) that the chat reads, all inside the customer's own prefix and keys.

CLAIMCLARITY · INGESTION & LAKE



In this diagram

Item	What it does	Details
Customer lab	Sends one received day of EDI files over SFTP.	
SFTP intake	AWS Transfer Family receives the files, and a copy step writes them into raw.	SFTP intake
S3 · raw	Holds the files as received, write-once and locked for 7 years.	S3 layout
EventBridge	Starts one parser task when Customer B's `_SUCCESS` marker lands (gates 1 and 2).	Parse trigger
Parser task	Parses the day on ECS Fargate as Customer B's parser role (gate 3), then exits.	Parser task

Item	What it does	Details
S3 · silver	Holds the lossless tables, one part file per table per batch.	S3 layout
S3 · gold	Holds the `sem.*` tables and PHI-stripped `lossless.*` tables, one version per batch.	Gold publish

How it works

1. The lab uploads a day's files to Transfer Family. A copy step writes them into `raw/received_date=YYYY-MM-DD/` and writes `_SUCCESS` last.
2. Customer B's EventBridge rule matches the marker and starts the parser task.
3. The task takes Customer B's batch lease, parses the files it has not loaded yet, and quarantines bad ones.
4. It writes silver, with `edi.source_file` last as the commit record.
5. It rebuilds the touched claims in gold, writes a new manifest and moves `CURRENT` to it.
6. It releases the lease and exits. Chat sessions read gold through `CURRENT`.

Specification

Item	Value
Isolation	Each customer has its own prefix, roles (SFTP copy, parser, reader, transcripts, audit) and KMS keys (lake, transcripts), plus its own EventBridge rule and parser task definition
Data held / PHI	Raw, silver and quarantine: full PHI. Gold: no direct patient identifiers.
Encryption	SSE-KMS with the customer's lake key on every lake object
Raw retention	Object Lock COMPLIANCE, 2,555 days; never deleted
One batch at a time	A lease object per customer (`staging/LOCK`)
Failure	A bad file is quarantined and the rest loads. A failed batch commits nothing and is re-run.
Logging	CloudTrail for every S3, KMS, ECS and IAM call; `edi.ingest_run` and `edi.source_file` rows per batch; run summary in CloudWatch Logs

Pages in this area

- SFTP intake · S3 layout · Parse trigger · Parser task · Gold publish

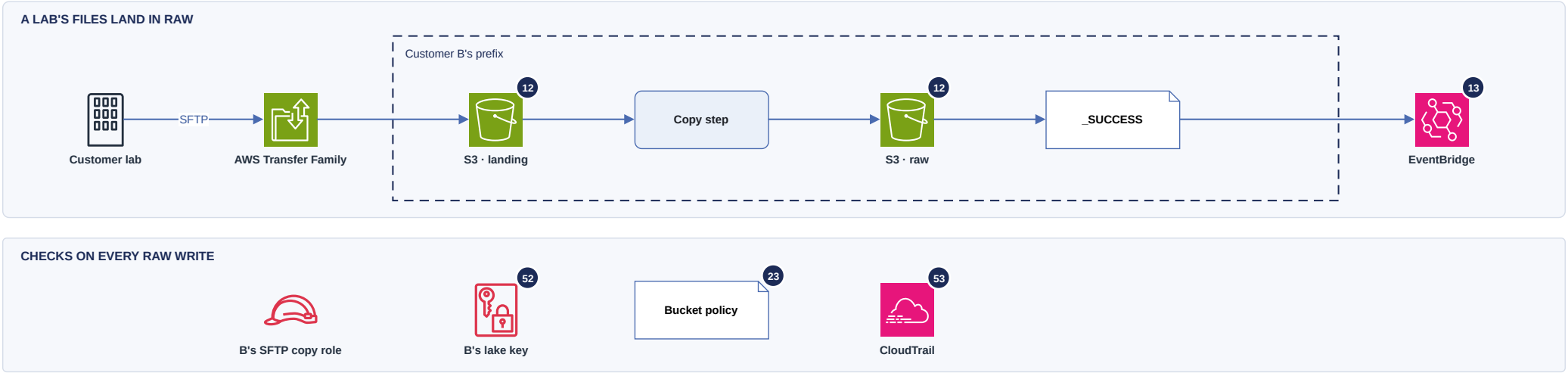
Related

Up: Overview · See also: Customer isolation · Encryption keys · Audit logging

SFTP intake

Labs deliver EDI files over SFTP to AWS Transfer Family, which lands them in the customer's `landing/` prefix. A copy step then writes each file into `raw/` as create-only, locked for 7 years and encrypted with the customer's lake key, and writes `_SUCCESS` last to start the parser.

CLAIMCLARITY · SFTP INTAKE



In this diagram

Item	What it does	Details
Customer lab	Uploads one received day over SFTP with the customer's SSH key.	
AWS Transfer Family	SFTP server; the customer's user can only write to its own `landing/`.	
S3 · landing	Holds uploaded files until the copy step has moved them.	S3 layout

Item	What it does	Details
Copy step	Copies the day into `raw/` as Customer B's SFTP copy role.	
S3 · raw	Keeps the files as received, write-once.	S3 layout
`_SUCCESS`	Empty marker the copy step writes last; it means the day is complete.	
EventBridge	Starts the parser for this customer.	Parse trigger
B's SFTP copy role	The only identity that writes Customer B's raw files.	
B's lake key	KMS key that encrypts every lake object of Customer B.	Encryption keys
Bucket policy	Refuses any raw write that is not create-only, locked and under the owner's key.	Customer isolation
CloudTrail	Records every write and key use, allowed or denied.	Audit logging

How it works

1. The lab connects to Transfer Family (SSH, port 22) and uploads into `received_date=YYYY-MM-DD/837/`, `835/` and `277/` under its `landing/` folder, then an empty `_SUCCESS`.
2. The lab's `_SUCCESS` in `landing/` starts the copy step for that day.
3. The copy step writes each file to the same path under `raw/`, create-only, with Object Lock and Customer B's lake key.
4. It writes `raw/…/_SUCCESS` last, then deletes the day from `landing/`.
5. S3 sends the Object Created event for `raw/…/_SUCCESS` to EventBridge, which starts the parser.

Specification

Permissions

Who	May	May not
Customer B's Transfer Family user	Upload to Customer B's `landing/`	See any other folder

Who	May	May not
Customer B's SFTP copy role	Read and delete `landing/`; `s3:PutObject` and `s3:PutObjectRetention` on `raw/`	Overwrite or delete `raw/`; touch any other customer
Customer B's lake key	Encrypt and decrypt for Customer B's lake roles, through S3 only	Be used by any other principal

Raw write rules (bucket policy)

Rule	Value
Transport	TLS only
Encryption	SSE-KMS with the key in the writer's `kms_key_arn` tag
Create-only	`If-None-Match: *` required
Retention	Object Lock COMPLIANCE, at least 2,555 days remaining
Deletes	Denied for everyone

Data, limits and failure

Item	Value
Data held / PHI	Full PHI in `landing/` and `raw/`
A file name that already exists in raw	Refused; the lab sends a corrected file under a new name, and both are kept
A day without the lab's `_SUCCESS`	Nothing is copied or parsed
A refused raw write	The copy step stops before `raw/…/_SUCCESS`; the day is re-run
Logging	CloudTrail S3 data events and KMS calls; Transfer Family session logs (user, file names, no contents) in CloudWatch Logs

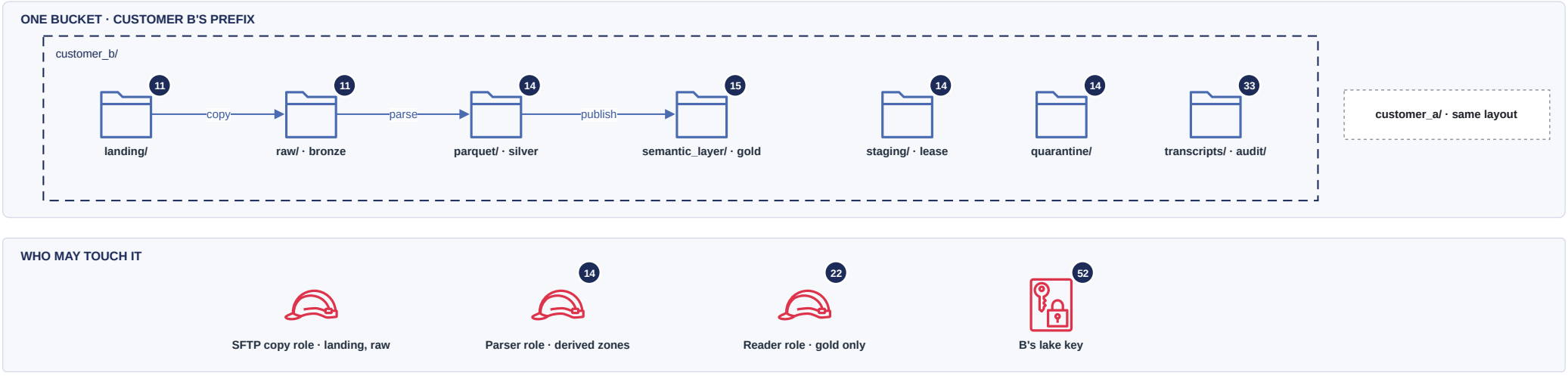
Related

Up: Ingestion & lake · Next: Parse trigger · See also: S3 layout

S3 layout

One S3 bucket, the ClaimClarity bucket, holds all customer data, with one prefix per customer. Files move through the prefix from `landing/` to `raw/`, `parquet/` (silver) and `semantic_layer/` (gold), and each zone is open only to the role that needs it.

CLAIMCLARITY · S3 LAYOUT



In this diagram

Item	What it does	Details
`landing/`	Holds SFTP uploads until they are copied to raw.	SFTP intake
`raw/` · bronze	Keeps the EDI files as received, write-once, locked 7 years.	SFTP intake
`parquet/` · silver	Holds the lossless tables, by table and received month.	Parser task

Item	What it does	Details
`semantic_layer/` · gold	Holds `CURRENT`, one manifest per version and the gold data files.	Gold publish
`staging/` · lease	Holds `LOCK`, the customer's batch lease.	Parser task
`quarantine/`	Holds copies of files that failed to parse.	Parser task
`transcripts/` · `audit/`	Hold chat state and the query audit, under their own keys.	State
`customer_a/`	Every other customer has the same layout with its own roles and keys.	Customer isolation
SFTP copy role	Moves files from `landing/` into `raw/`.	SFTP intake
Parser role	Reads raw and writes the derived zones.	Parser task
Reader role	Reads gold only.	Token to credentials
B's lake key	Encrypts every lake object of Customer B.	Encryption keys

How it works

1. The SFTP copy role moves uploads from `landing/` into `raw/`.
2. The parser role reads `raw/`, takes the lease in `staging/`, quarantines bad files, writes `parquet/` and publishes `semantic_layer/`.
3. The reader role reads `semantic_layer/` and nothing else.

Specification

Keys

```
 <customer>/landing/received_date=YYYY-MM-DD/{837,835,277}/<file>
<customer>/raw/received_date=YYYY-MM-DD/{837,835,277}/<file>   + _SUCCESS
<customer>/staging/LOCK
<customer>/quarantine/<run_id>/<file>
<customer>/parquet/<table>/received_month=YYYY-MM/{part,compacted}-<run_id>.parquet
```

```
<customer>/semantic_layer/CURRENT
<customer>/semantic_layer/v=<run_id>/manifest.json
<customer>/semantic_layer/<relation>/<partition>/part-<run_id>-<n>.parquet
```

Permissions by zone

Zone	SFTP copy role	Parser role	Reader role
`landing/`	Get, Delete	–	–
`raw/`	Put, PutObjectRetention	Get, GetObjectVersion	–
`staging/`	–	Get, Put, Delete	–
`quarantine/`	–	Put, Delete	–
`parquet/`	–	Get, Put, Delete	–
`semantic_layer/`	–	Get, Put, Delete	Get, List

Zones, PHI and retention

Zone	PHI	Retention
`landing/`	Yes	Deleted after the copy
`raw/`	Yes	Object Lock COMPLIANCE 2,555 days; no deletes; no lifecycle rule
`staging/`	No	Old versions expire after 1 day
`quarantine/`	Yes	Kept for review
`parquet/`	Yes	Old versions expire after 7 days
`semantic_layer/`	No direct patient identifiers	Retired files deleted after 20 minutes; old versions expire after 1 day

Bucket settings

Setting	Value
Versioning, Object Lock	On. Retention is set per raw object, not as a bucket default.
Encryption	SSE-KMS. Writes must name the writer's own lake key; the bucket default key is refused for customer prefixes.
Bucket policy	One document for all customers: TLS only; principals without a `customer` tag denied; access only under `${aws:PrincipalTag/customer}/`; raw write-once and never deleted
Who may change bucket settings	Only the Terraform deploy role
Other buckets	The web app and the access logs have their own buckets
Logging	CloudTrail S3 data events for every object call

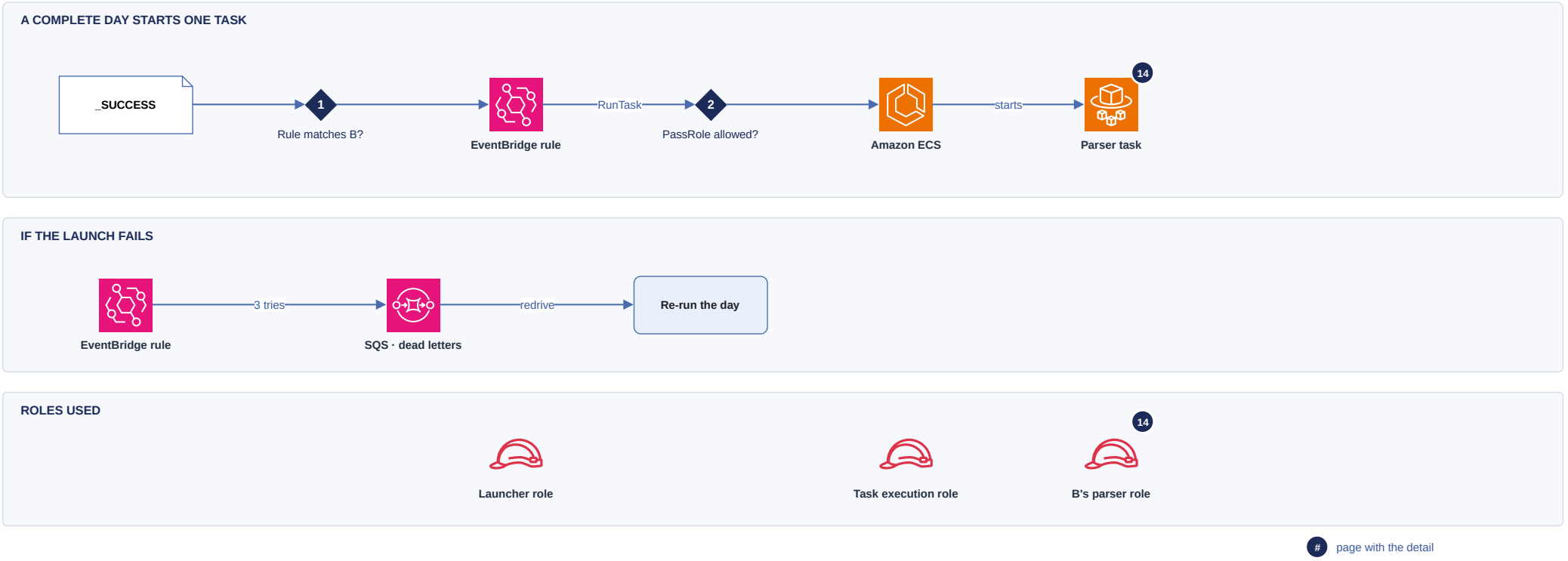
Related

Up: Ingestion & lake · See also: IAM and KMS matrix

Parse trigger

When a customer's day in `raw/` is complete, its `_SUCCESS` marker starts exactly one parser task for that customer. An EventBridge rule per customer matches the marker, and ECS starts a Fargate task from that customer's task definition; the launcher can start tasks but can never read data.

CLAIMCLARITY · PARSE TRIGGER



In this diagram

Item	What it does	Details
`_SUCCESS`	Marks a complete day in Customer B's `raw/`.	SFTP intake
Gate 1 · Rule matches B?	Only a key under Customer B's raw prefix matches Customer B's rule.	
EventBridge rule	Calls ECS RunTask as the launcher role.	
Gate 2 · PassRole allowed?	The launcher may pass only the parser roles, only to ECS tasks.	
Amazon ECS	Runs Customer B's parser task definition on Fargate.	
Parser task	Parses the day as Customer B's parser role.	Parser task
SQS · dead letters	Keeps events the rule could not deliver.	
Re-run the day	Starts the task again for that day; loaded files are skipped.	
Launcher role	The rule's target role: RunTask and PassRole only.	
Task execution role	Lets ECS pull the image and write container logs.	
B's parser role	The only credentials the code in the task gets.	Parser task

How it works

1. The `_SUCCESS` object lands; S3 sends an Object Created event to EventBridge.
2. Customer B's rule matches it (gate 1) and calls RunTask with `SUCCESS_KEY` and `EVENT_ID`.
3. ECS checks that the launcher may pass Customer B's parser role (gate 2) and starts the task on Fargate.
4. If RunTask fails 3 times, the event goes to the dead-letter queue and the day is re-run from there.

Specification

Item	Value
Rule pattern	`source = aws.s3`, `detail-type = Object Created`, the ClaimClarity bucket, key `<customer>/raw/received_date=*/_SUCCESS`
Target	ECS RunTask, ClaimClarity cluster, Fargate, private subnets, no public IP
Task environment	`CUSTOMER`, `BUCKET`, `KMS_KEY_ARN` fixed in the task definition; `SUCCESS_KEY`, `EVENT_ID` from the event
Retry policy	3 attempts, then the SQS dead-letter queue
Launcher role	Trusted by EventBridge in this account; `ecs:RunTask` on parser task definitions in the ClaimClarity cluster; `iam:PassRole` on the parser roles and the task execution role to `ecs-tasks.amazonaws.com`; no S3 or KMS access
Task execution role	Pull the parser image from ECR; write container logs
Parser role	Trusted by ECS tasks of this account; sessions up to 1 hour
Task credentials	They can outlive the task (up to about 6 hours). The boundary is the role's permissions: Customer B's prefix only.
Duplicate event	A second task waits for the lease, finds every file loaded and exits with status `noop`
Task fails after start	No EventBridge retry; the task stops with a non-zero exit code; re-run the day
Logging	CloudTrail: RunTask, PassRole, failed invocations. `EVENT_ID` and the task ARN are stored in `edi.ingest_run`.

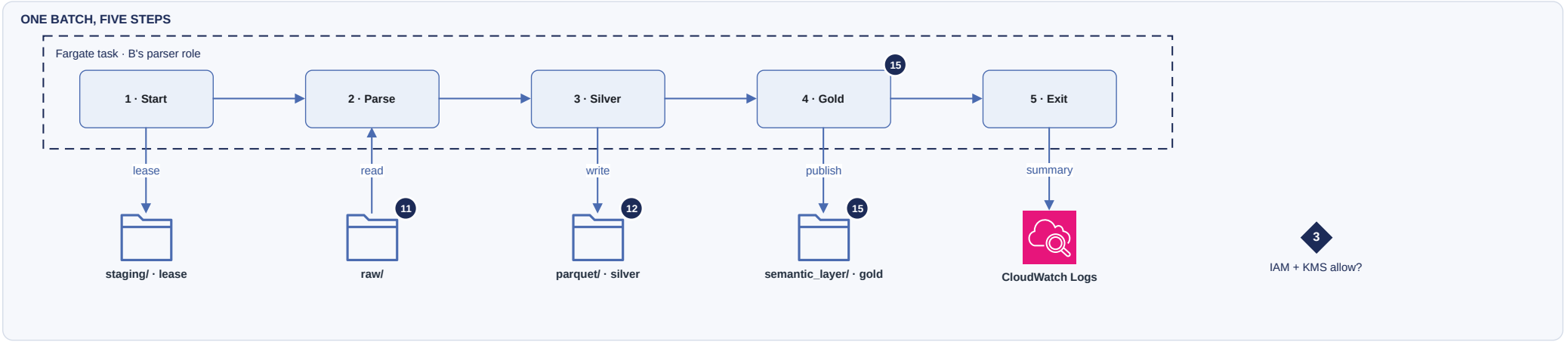
Related

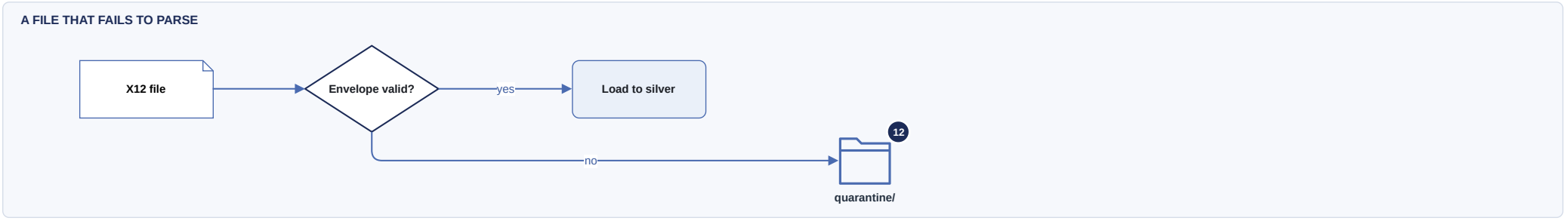
Up: Ingestion & lake · Previous: SFTP intake · Next: Parser task

Parser task

The parser task turns one customer's received day into silver and gold, on ECS Fargate as that customer's parser role, and exits. A lease allows one batch per customer at a time, and a commit record makes every batch all-or-nothing.

CLAIMCLARITY · PARSER TASK





page with the detail

In this diagram

Item	What it does	Details
1 · Start	Takes Customer B's batch lease and deletes leftovers of crashed batches.	
`staging/` · lease	Holds the lease object `LOCK`.	S3 layout
2 · Parse	Reads the day's new raw files by version and parses them inside the task.	
`raw/`	The source files; the parser only reads them.	SFTP intake
3 · Silver	Writes one part file per table, then `edi.source_file` as the commit record.	
`parquet/` · silver	The lossless tables.	S3 layout
4 · Gold	Publishes a new gold version, then compacts silver months.	Gold publish
`semantic_layer/` · gold	Gold files, manifests and `CURRENT`.	Gold publish
5 · Exit	Releases the lease and logs a run summary; the container is destroyed.	
CloudWatch Logs	Keeps the run summary.	Monitoring
Gate 3 · IAM + KMS allow?	Every S3 call needs the parser role's IAM permission and the lake key's permission.	
X12 file → Envelope valid?	A file with a broken envelope is not loaded.	
Load to silver	A valid file is staged and loaded.	
`quarantine/`	Keeps a copy of each rejected file.	S3 layout

How it works

1. **Start:** take `staging/LOCK`; delete part files of runs that never committed.
2. **Parse:** list the day folder; skip files whose key and version are already in `edi.source_file`; stage rows in DuckDB under the lossless schema. A broken file gets a `REJECTED` row and a copy in `quarantine/<run_id>/`.
3. **Silver:** renew the lease; write `parquet/<table>/received_month=YYYY-MM/part-<run_id>.parquet`; write `edi.source_file` last.

4. **Gold:** renew the lease; publish the new version (Gold publish); compact silver.
5. **Exit:** release the lease, log the summary, exit.

Specification

Task and permissions

Item	Value
Size	4–8 GB memory for 5 MB of raw EDI per day; `PARSE_WORKERS` = task vCPUs
Storage	Fargate ephemeral storage, removed with the task; staged rows spill to disk above 256 MB
Settings	`GOLD_MODE` (`incremental` or `full`), `PARSE_WORKERS`, `SILVER_OPEN_MONTH_MAX_PARTS` (4)
List	The ClaimClarity bucket, prefix `<customer>/` only
Read	`raw/`, `staging/`, `parquet/`, `semantic_layer/` (Get, GetObjectVersion)
Write	`staging/`, `parquet/`, `semantic_layer/`, `quarantine/` (Put, Delete)
Key	Customer B's lake key: Decrypt and GenerateDataKey through S3, for objects under `<customer>/` only
Never	Write or delete `raw/`; touch another prefix or key

Lease and fencing

Item	Value
Take	Create `staging/LOCK` with `If-None-Match: *`, or take an expired one over with `If-Match`
Wait	Up to 15 minutes, then the task fails
Lease	2 hours
Renew	`If-Match` on its own ETag before the commit record, compaction and `CURRENT`
Fencing	A failed renew stops the task (`LockLost`) before that write

Item	Value
Release	Overwrite with an expired lease, `If-Match`; never removes another holder's lease

Compaction

Partition	When	Result
Closed month	Once, after gold includes every committed run	One `compacted-<run_id>.parquet` per table
Open month	When a table gold reads has 4 daily parts	The same

Failure behaviour

Event	What happens
Broken envelope (no ISA, missing trailer, mismatched control numbers, segment outside a transaction)	File rejected and quarantined; the rest loads
Wrong SE01 count	Loaded and flagged
Any other error	Batch fails before its commit record; nothing is visible; re-run the day
Lease busy 15 minutes	Task fails without writing; re-run the day
Crash after the silver commit	The next batch catches gold up

Data held and logging

Item	Value
PHI	Full PHI in memory and on task storage while it runs; written only to `parquet/` and `quarantine/`
`edi.ingest_run`	Run id, times, parser version, files seen / loaded / rejected, task ARN, event id
`edi.source_file`	Per file: S3 key, version id, ETag, size, SHA-256, KMS key, parse status, messages

Item	Value
Run summary	CloudWatch Logs: status, counts, timings; no PHI
API calls	CloudTrail, allowed or denied

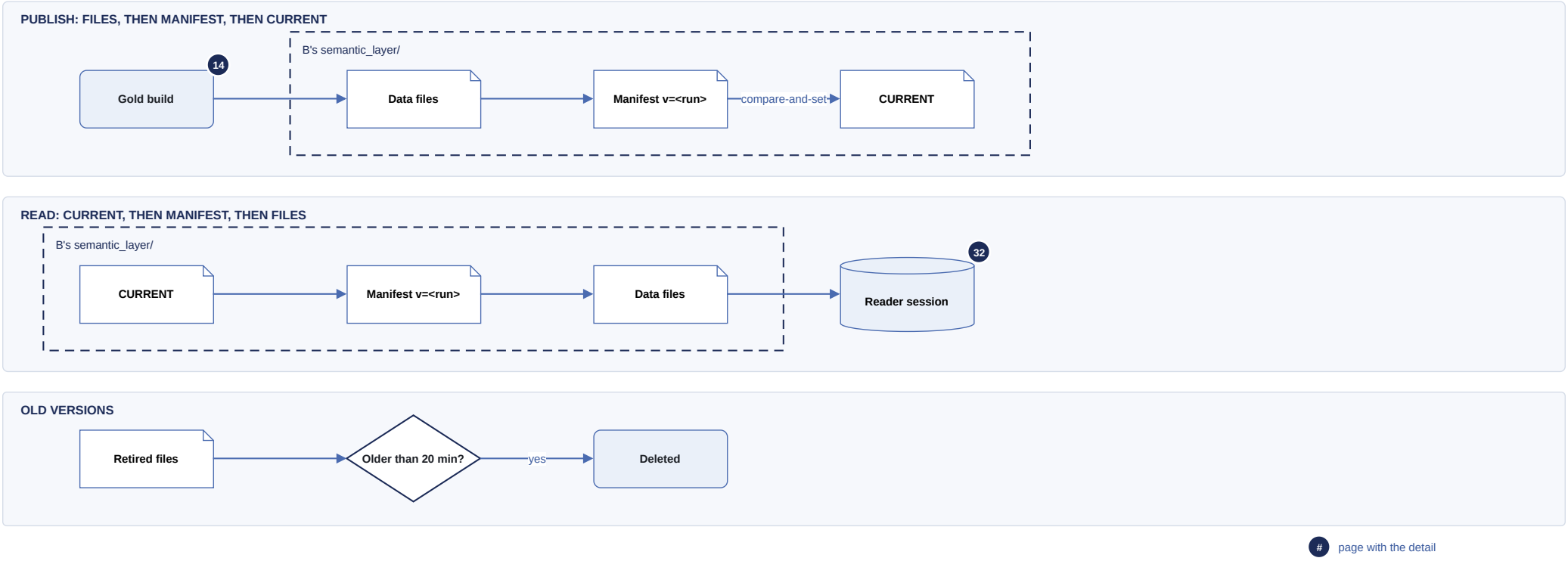
Related

Up: Ingestion & lake · Previous: Parse trigger · Next: Gold publish

Gold publish

Each batch publishes a new gold version that readers can never see half-built: new data files first, then a manifest that lists them, then a compare-and-set on the `CURRENT` pointer. Readers follow `CURRENT` to one manifest and read exactly its files, and retired files stay for 20 minutes so open sessions finish.

CLAIMCLARITY · GOLD PUBLISH



In this diagram

Item	What it does	Details
Gold build	Step 4 of the parser task: recomputes the claims the batch touched.	Parser task
Data files	New Parquet files; a file is never overwritten.	
Manifest `v=<run>`	Lists every file of every relation in this version.	
`CURRENT`	Names the live version; moved only by compare-and-set.	
Reader session	DuckDB in a run worker, as Customer B's reader role, reading exactly the listed files.	Chat turn
Retired files	Files a newer version no longer lists.	
Older than 20 min?	Retired files are deleted only after every reader that saw them has expired.	
Deleted	The build removes them.	

How it works

1. The build reads `CURRENT` and its ETag, and the manifest it names.
2. It loads the new silver runs plus the older rows of the claims they touch, and runs the semantic view SQL.
3. It writes new data files, then `v=<run_id>/manifest.json`.
4. It writes `CURRENT` with `If-Match` on the ETag from step 1. If another build won, it stops.
5. It deletes files retired more than 20 minutes ago.
6. A reader reads `CURRENT`, then that manifest, then the listed files.

Specification

Item	Value
Relations	10 `sem.*` relations; 15 PHI-stripped `lossless.*` tables for audited escalation, published next to gold

Item	Value
Partitions	Claim relations by cohort month (3 newest months, older months by quarter); small relations one file each
Format	Parquet, ZSTD, 1,048,576-row groups; a zero-row schema file listed first per relation
Incremental build	Default: only the touched claims are recomputed and their partitions rewritten
Full build	`GOLD_MODE=full`, the first build, a change to view SQL or reference data, or a late batch for an older month
Month close	Partitions re-tiered (months beyond the 3 newest fold into quarters); orphan files swept
`CURRENT`	One object, `v=<run_id>`; first write `If-None-Match: *`, then `If-Match`; only moves forward
Retention	Retired files deleted after 1,200 s (900 s reader credentials + 300 s); noncurrent versions expire after 1 day
Permissions	Parser role: Get, Put, Delete on `semantic_layer/`. Reader role: Get and List only.
Encryption	Customer B's lake key
PHI	No direct patient identifiers: person names, ids, addresses, birth dates and bank accounts are removed
Crash before `CURRENT`	Readers keep the old version; the new files are swept later
Logging	Gold build report in the run summary (CloudWatch Logs); CloudTrail for every write to `CURRENT`

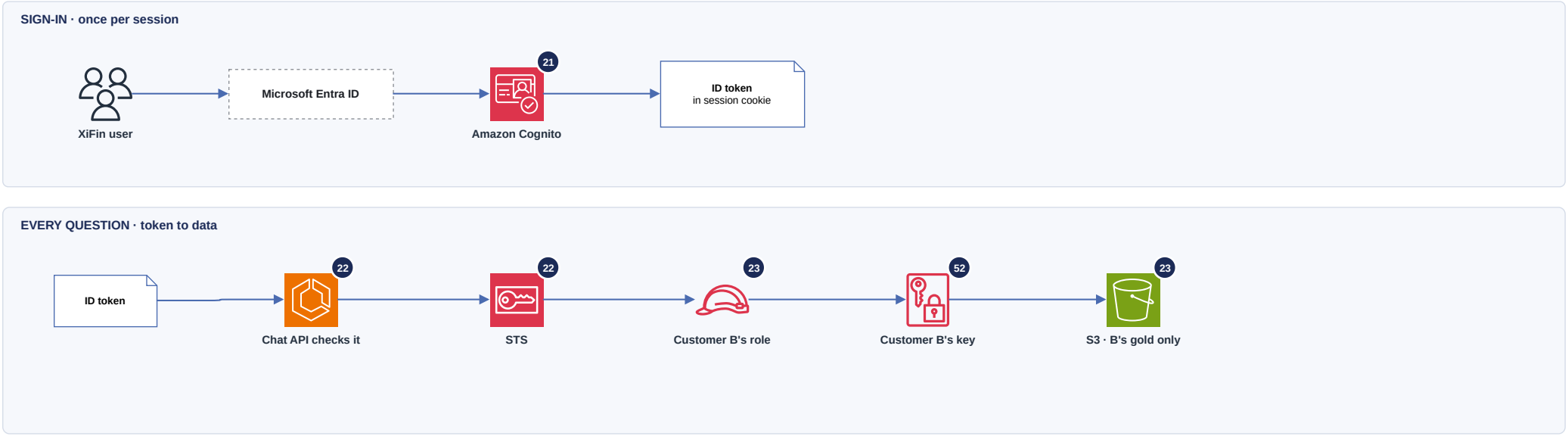
Related

Up: Ingestion & lake · Previous: Parser task · See also: Chat turn · SQL guard and escalation

Identity & access

A XiFin user signs in with Microsoft Entra ID, and Amazon Cognito issues an ID token that names exactly one customer. On every question, STS exchanges that token for 15-minute credentials of the customer's reader role, and those credentials can read only that customer's gold data.

CLAIMCLARITY · IDENTITY & ACCESS



In this diagram

Item	What it does	Details
XiFin user	Opens the ClaimClarity web app and signs in.	Sign-in

Item	What it does	Details
Microsoft Entra ID	Company login. Issues a token only to members of the groups assigned to the ClaimClarity app.	Sign-in
Amazon Cognito	Federates Entra ID, maps the user to one customer and issues the ID token.	Sign-in
ID token	Names the customer. Kept in an httpOnly session cookie for 60 minutes.	Sign-in
Chat API checks it	Verifies the token on every request (gate 4).	Token to credentials
STS	Exchanges the token for 15-minute credentials if the role's trust policy allows it (gate 5).	Token to credentials
Customer B's role	The reader role the credentials belong to. It reads Customer B's gold prefix and nothing else.	Customer isolation
Customer B's key	Decrypts Customer B's objects, only for Customer B's roles, only through S3 (gate 6).	Encryption keys
S3 · B's gold only	The current version of Customer B's gold tables.	Customer isolation

How it works

1. The user signs in at Entra ID. Users outside the assigned groups are refused.
2. Cognito refuses users with no customer mapping, and issues a 60-minute ID token for the others.
3. The chat API stores the token in the `cc_session` cookie and verifies it on every request (gate 4).
4. A run worker exchanges the token at STS for Customer B's reader role (gate 5). The credentials last 15 minutes and are renewed with the same token in long chats.
5. Each read of Customer B's gold prefix needs IAM, the bucket policy and Customer B's key to allow it (gate 6).

Specification

Item	Value
Identity provider	Microsoft Entra ID, one enterprise app with "Assignment required" on
Federation	Cognito user pool with Entra ID as its only identity provider
Customer claim	`custom:customer`, set by an administrator, not writable by the user

Item	Value
ID token	60 minutes. The user signs in again when it expires
Session cookie	`cc_session` · HttpOnly · Secure · SameSite=Strict · Path=/api · Max-Age 3600
Reader credentials	AssumeRoleWithWebIdentity, 900 s, renewed when fewer than 3 minutes remain
Reader scope	List and read `<customer>/semantic_layer/`, current object versions only
The customer's roles and keys	Parser, reader, SFTP copy, transcript and audit roles; a lake key and a transcript key. See IAM and KMS matrix
Logging	Entra sign-in log; CloudTrail for every STS, S3 and KMS call. Tokens are never logged

Pages in this area

- Sign-in: Entra ID, Cognito and the session cookie
- Token to credentials: gates 4, 5 and 6
- Customer isolation: why Customer A's session cannot reach Customer B's data

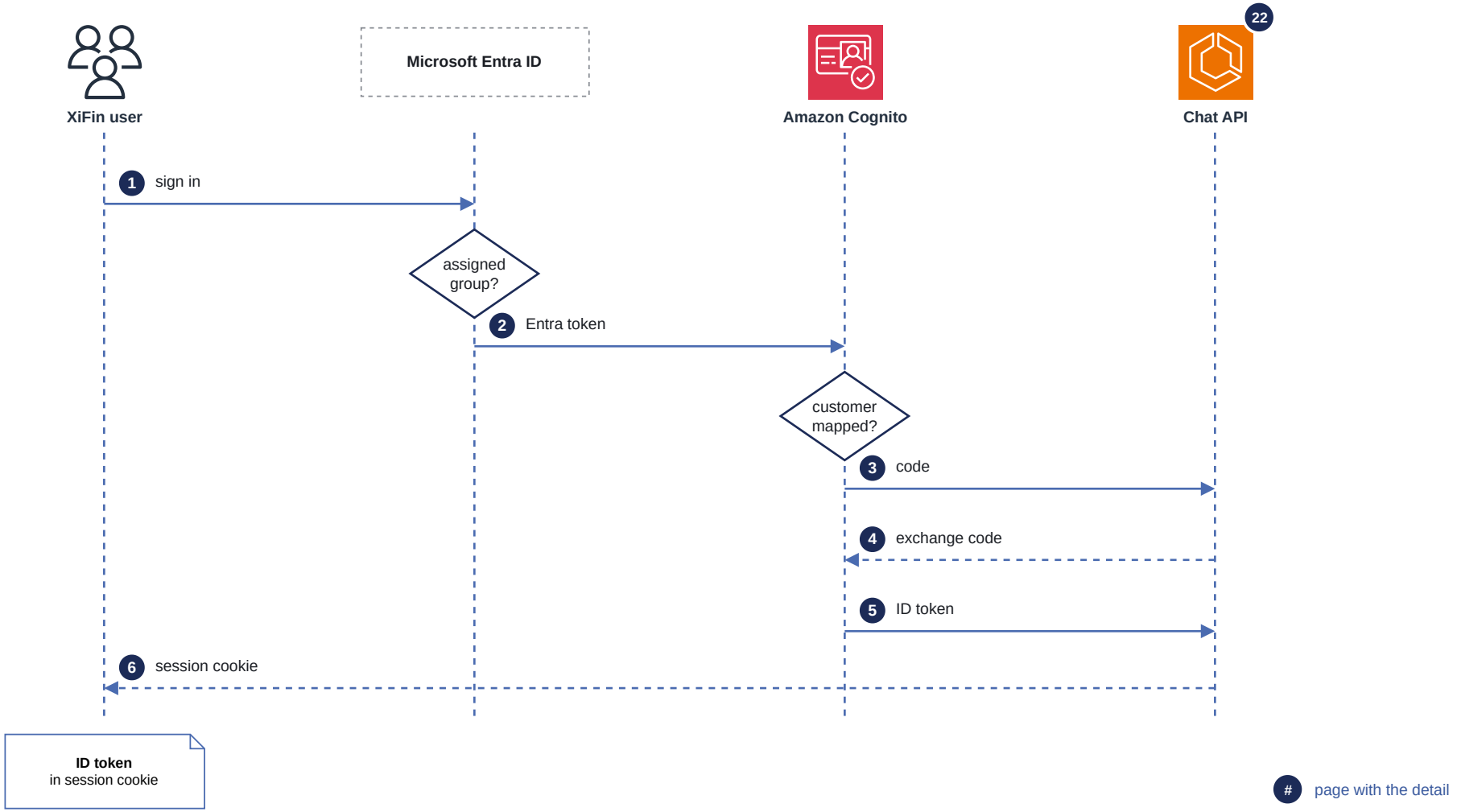
Related

Up: Overview · See also: Network & security · IAM and KMS matrix

Sign-in

Entra ID decides whether a person may use ClaimClarity at all, and Cognito decides which one customer they work for. The result is a Cognito ID token that the chat API keeps in an httpOnly cookie, so no script in the page can read it.

CLAIMCLARITY · SIGN-IN



In this diagram

Item	What it does	Details
XiFin user	Opens the app in a browser with no valid session and is sent to sign in.	

Item	What it does	Details
Microsoft Entra ID	Company login and MFA. Signs in only members of the assigned groups ("assigned group?").	Entra ID card
Amazon Cognito	Verifies the Entra token, refuses users with no customer ("customer mapped?") and issues the ID token.	Cognito card
Chat API	Exchanges the authorization code for the ID token and sets the session cookie.	Token to credentials
ID token	Held in the `cc_session` cookie and sent with every API call.	

How it works

1. The user signs in at Entra ID. Entra issues a token only if the user is in a group assigned to the app.
2. Cognito verifies the Entra token (issuer, tenant, client id) and creates the Cognito user on first sign-in. It refuses a user that no administrator has mapped to a customer.
3. Cognito adds the session tags `customer` and `user` (the `sub`) to the token, then sends an authorization code to the chat API's sign-in callback.
4. The chat API exchanges the code at the Cognito token endpoint, using the app client secret from Secrets Manager.
5. Cognito returns the ID token. The chat API verifies it (gate 4).
6. The chat API sets the `cc_session` cookie and sends the user back to the app.

Specification

ID token

Claim	Value
`iss`, `aud`	The ClaimClarity user pool and app client
`token_use`	`id`
`sub`	Cognito user id. Conversations are keyed by customer and `sub`
`custom:customer`	The one customer the user works for
`https://aws.amazon.com/tags`	Session tags `customer`, `user` and, for customer admins, `transcript_admin`
`exp`	3600 s after issue. Signed RS256

Session cookie

Attribute	Value
Name and value	`cc_session` = the ID token
Flags	HttpOnly · Secure · SameSite=Strict · Path=/api · Max-Age 3600
Writes	Cookie-borne POST, PUT, PATCH and DELETE need the header `x-cc-csrf: 1`, or the API answers 403
Never	In a URL or in browser storage

Lifetimes and revocation

Item	Lifetime	To cut it off
Entra session	Company policy	Disable the account or remove it from the assigned groups
ID token	60 minutes	It stays valid until it expires. Disable the Cognito user to stop new sign-ins
Refresh token	30 days, kept by Cognito, never given to the browser	Global sign-out
Reader credentials	15 minutes	"Revoke active sessions" on the customer's reader role

Sign-out (`POST /api/auth/sign-out`) revokes the user's Cognito tokens, expires the cookie and sends `Clear-Site-Data: "cache", "storage"`.

Failure behaviour

Case	Result
Not in an assigned group	Entra refuses (AADSTS50105). No token
No customer mapping	Cognito refuses. No token
Expired cookie	401 `auth_expired`. The app sends the user through sign-in once

Logging

Entra keeps its sign-in log. Cognito API calls are in CloudTrail. The API records `signed_out` with customer and user only. Tokens are never logged.

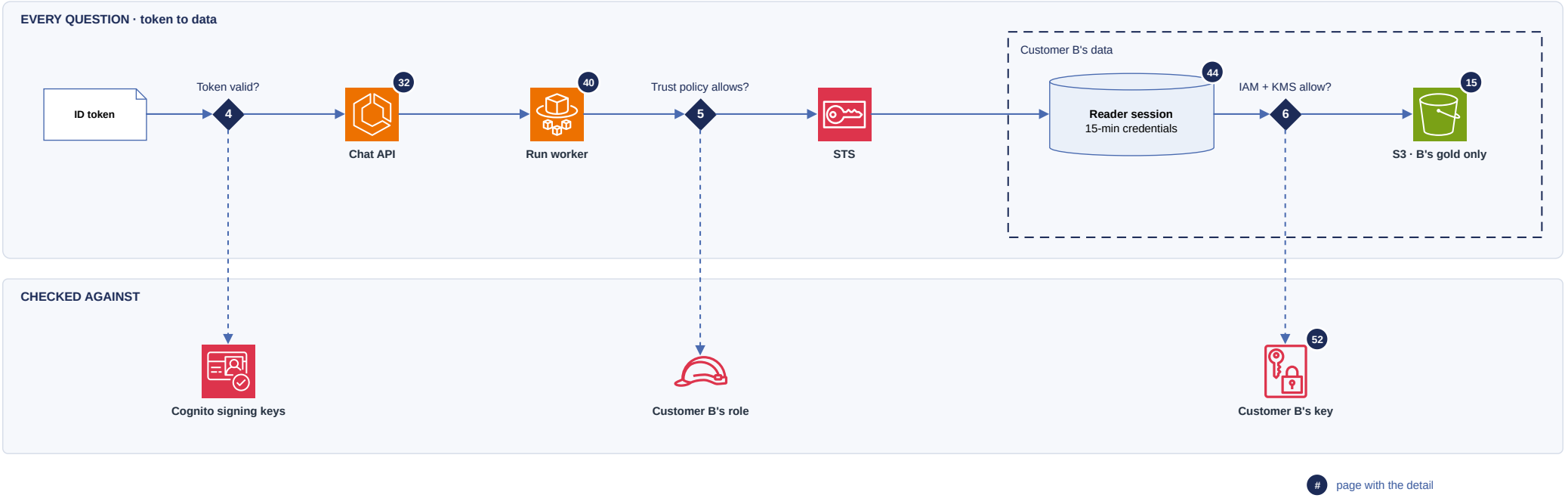
Related

Up: Identity & access · Next: Token to credentials · See also: Network

Token to credentials

Every question passes three AWS checks before any customer data is read: the token (gate 4), the role's trust policy (gate 5) and IAM plus KMS on every read (gate 6). The run worker has no standing access to customer data: it gets 15-minute credentials for one customer's reader role, for one run.

CLAIMCLARITY · TOKEN TO CREDENTIALS



In this diagram

Item	What it does	Details
ID token	Arrives in the `cc_session` cookie with each API call.	Sign-in

Item	What it does	Details
Gate 4 · Token valid?	Checks the token against the Cognito signing keys.	
Chat API	Takes the customer from the verified token only, then queues the run.	Chat turn
Run worker	Holds the verified token for the run and calls STS.	Analyzer
Gate 5 · Trust policy allows?	Customer B's role accepts only ClaimClarity tokens tagged `customer = B`.	
STS	Returns 15-minute credentials for Customer B's reader role.	STS card
Reader session	DuckDB session opened with those credentials, limited to B's gold relations.	SQL guard & escalation
Gate 6 · IAM + KMS allow?	Each S3 read must pass B's role policy, the bucket policy and B's key policy.	
S3 · B's gold only	The version of `B/semantic_layer/` that `CURRENT` names.	Gold publish
Cognito signing keys	Public keys the chat API verifies the token with. Cached.	Network
Customer B's role	The reader role and its trust policy.	IAM and KMS matrix
Customer B's key	Decrypts B's objects, only for B's roles, only through S3.	Encryption keys

How it works

1. **Gate 4.** The chat API checks the token: RS256 with a key from the Cognito signing keys, issuer, audience (the ClaimClarity app client), expiry, `token_use = id` and a `custom:customer` claim. Any failure is a 401.
2. The API takes the customer and user from the token. Nothing in the body, query string or headers can change them. It queues the run.
3. **Gate 5.** The run worker calls AssumeRoleWithWebIdentity for Customer B's reader role, with session name `u-<sub prefix>-<request id>` and a duration of 900 s. STS verifies the token again and evaluates the trust policy.
4. The worker opens the reader session: it reads `CURRENT` and that version's manifest, then mounts one view per gold table, each with its own scoped secret and allowed directory. External access is off and the configuration is locked.
5. **Gate 6.** Every S3 read needs B's role policy, the bucket policy and B's key policy to allow it.
6. In long chats, the worker renews the credentials with the same token when fewer than 3 minutes remain, on the same gold version. SQL is never run on expired credentials.

Specification

Trust policy of Customer B's reader role (gate 5)

Element	Value
Principal	The IAM OIDC provider for the ClaimClarity Cognito user pool
Actions	`sts:AssumeRoleWithWebIdentity`, `sts:TagSession`
Condition	`<pool>:aud` = the ClaimClarity app client, and `aws:RequestTag/customer` = B
Session length	900 s requested; the role's maximum is 3600 s

Customer B's reader role policy (gate 6, IAM part)

Allowed	On
`s3:ListBucket`	The ClaimClarity bucket, prefix `B/semantic_layer/*` only
`s3:GetObject`	`B/semantic_layer/*`

No `GetObjectVersion`, no raw or silver, no transcripts, no writes. `lossless.*` tables are PHI-stripped copies published next to gold, so even an escalated session never reads a silver object.

Limits

Item	Value
Credentials	900 s, renewed when fewer than 180 s remain
ID token	3600 s. No renewal after it expires: the run ends and the user signs in again
Signing keys	Cached 24 h; refetched on an unknown key id, at most every 5 minutes
Reader session	2 GB memory, no temp directory, configuration locked

Failure behaviour

Failure	Result
Gate 4	401 `auth_expired`. No run is queued
Gate 5	The run ends with a sign-in-again message. CloudTrail has the denied call
Gate 6	The SQL call fails and the agent audit records `outcome=error`
Credentials expired at a SQL call	Refused and audited as `expired`
STS throttled	3 retries with backoff, then the run fails

Logging

CloudTrail records every AssumeRoleWithWebIdentity, S3 GetObject and KMS Decrypt, allowed or denied. The agent audit records the session opening (customer, user, gold version, escalation flag, a hash of the token) and every SQL call. See Audit logging.

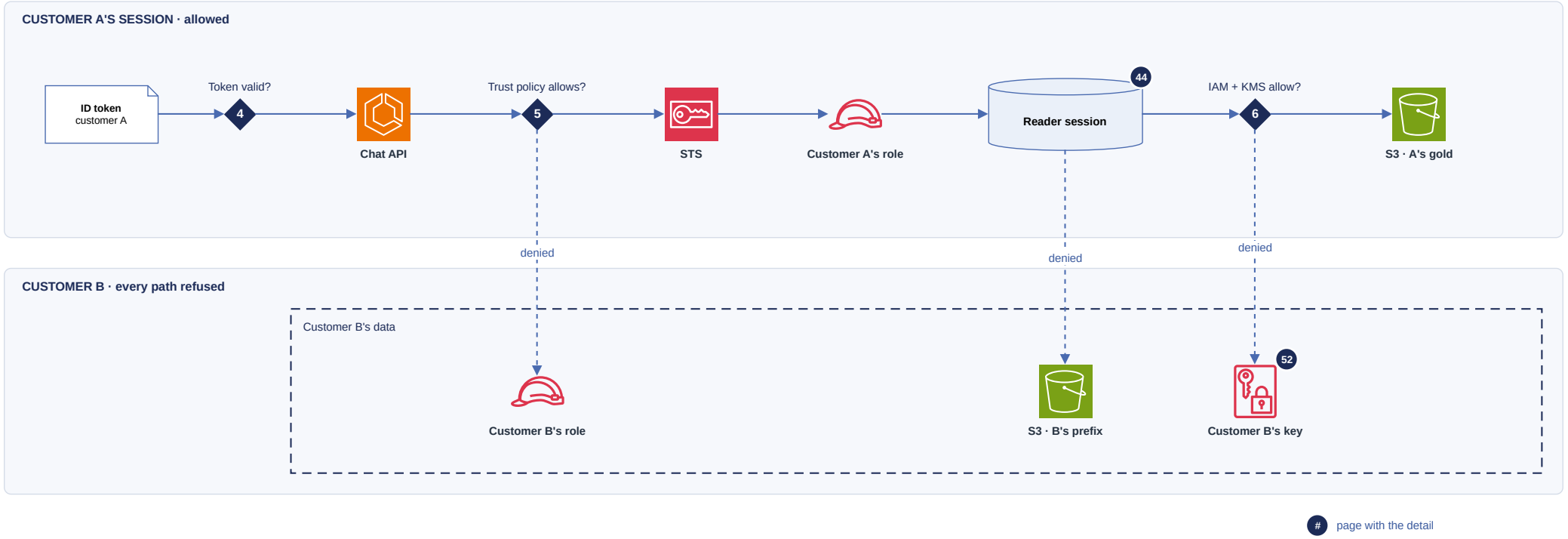
Related

Up: Identity & access · Previous: Sign-in · Next: Customer isolation

Customer isolation

Customer A's and Customer B's data sit in the same ClaimClarity bucket, separated by independent layers that each refuse on their own. A Customer A session follows its own path to A's gold, and every branch toward Customer B is denied.

CLAIMCLARITY · CUSTOMER ISOLATION



In this diagram

Item	What it does	Details
ID token · customer A	Signed by Cognito with `custom:customer = A` and the session tag `customer = A`.	Sign-in
Gate 4 · Token valid?	The chat API takes the customer from the verified token only.	Token to credentials
Chat API	Ignores any customer named in the request body, URL or headers.	
Gate 5 · Trust policy allows?	Only A's roles trust a token tagged `customer = A`. B's role refuses it.	
STS	Returns credentials for Customer A's reader role only.	
Customer A's role	Reads `A/semantic_layer/` only.	IAM and KMS matrix
Reader session	Its SQL guard and allowed directories cover A's gold relations only.	SQL guard & escalation
Gate 6 · IAM + KMS allow?	A's role policy, the bucket policy and the key policy decide each read.	
S3 · A's gold	The only data the session can read.	
Customer B's role	Refuses A's token at gate 5.	
S3 · B's prefix	Unreachable: the reader session refuses the path, and IAM and the bucket policy deny the read.	
Customer B's key	Refuses to decrypt for any role that is not Customer B's.	Encryption keys

How it works

1. A request that names Customer B still runs as Customer A: the customer comes from the verified token (gate 4). Conversations are keyed by customer and `sub`, so B's conversations are not found either.
2. A's token cannot assume B's role. B's trust policy requires `aws:RequestTag/customer = B`, and Cognito writes that tag from an attribute only an administrator can set (gate 5). A forged token fails the signature check.
3. A's credentials cannot read B's objects (gate 6). A's role policy allows `A/semantic_layer/*` only. The bucket policy denies object access outside `${aws:PrincipalTag/customer}/`. B's key decrypts only for B's roles.

4. SQL that names a path under B's prefix never runs. The SQL guard rejects file and network functions, DuckDB's allowed directories list only A's gold relations, and the configuration is locked.

Specification

Layer	Checks	Set in	Refusal
Entra ID	User is in an assigned group	Entra enterprise app	AADSTS50105
Cognito	User has a customer mapping	Cognito user attribute, administrators only	No token
Chat API (gate 4)	Signature, issuer, audience, expiry, customer claim	API configuration	401
STS (gate 5)	Cognito provider, app client, `customer` tag	Each reader role's trust policy	`AccessDenied`
IAM role policy (gate 6)	`<customer>/semantic_layer/*`, current version only	Each reader role	`AccessDenied`
Bucket policy (gate 6)	Object access only under the caller's `customer` tag; principals without the tag denied	One policy for all customers	`AccessDenied`
KMS key policy (gate 6)	The customer's roles only, through S3, under that customer's prefix	Each customer's keys	`AccessDenied`
Reader session	SQL guard, allowed directories, scoped secrets, locked configuration	Opened per run	Guard error
Chat state	Keyed by customer and `sub`	Chat API	404

To take a customer offline at once, disable that customer's keys: every one of its objects becomes unreadable. A key admin can disable a key but cannot decrypt with it.

Logging. Refusals at gates 5 and 6 are CloudTrail events with `errorCode = AccessDenied` and raise the cross-customer denial alarm. Guard refusals are agent-audit records with `outcome = blocked`. See Audit logging.

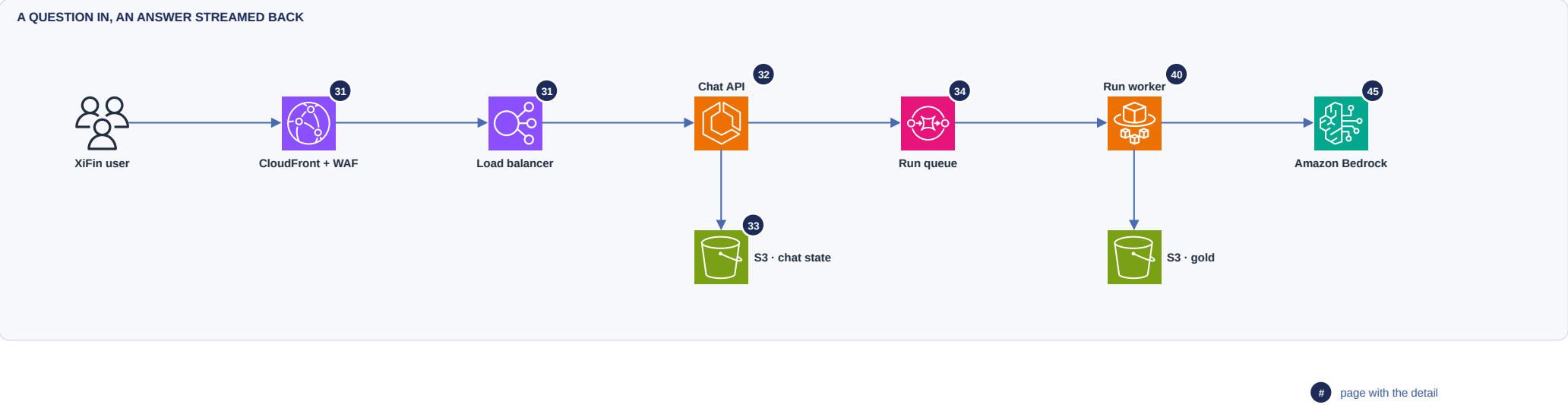
Related

Up: Identity & access · Previous: Token to credentials · See also: Encryption keys · IAM and KMS matrix

Chat backend

The chat API takes a question, queues a run and streams the answer back. A run worker on Fargate does the work. All chat state is kept in S3 objects under the customer's prefix, and there is no database.

CLAIMCLARITY · CHAT BACKEND



In this diagram

Item	What it does	Details
XiFin user	Asks a question in the React chat app and reads the answer as a stream	
CloudFront + WAF	Serves the web app, filters every request, and passes `/api/*` on	Edge
Load balancer	Spreads API calls across the chat API tasks	Edge

Item	What it does	Details
Chat API	Checks the token and limits, saves the question, queues the run, streams the events	Chat turn
S3 · chat state	Conversations, run-event chunks and cancel flags under the customer's prefix	State
Run queue	Amazon SQS. Holds one message per admitted run.	Limits
Run worker	Runs the analyzer for one question with the customer's reader credentials	Analyzer
S3 · gold	The customer's `sem.*` tables, read-only	Gold publish
Amazon Bedrock	Claude Haiku, called by the run worker	Model calls

How it works

1. The browser sends the question to `POST /api/chat` through CloudFront, WAF and the load balancer.
2. The chat API checks the token and the limits, saves the question and puts a run on the queue.
3. A run worker takes the run and exchanges the user's token for the customer's reader credentials.
4. The analyzer in the worker reads gold and calls Claude Haiku on Amazon Bedrock.
5. The worker writes every event to S3. The chat API streams the events to the browser as server-sent events.
6. The worker saves the answer in the conversation.

Specification

Item	Value
Compute	Two ECS services on Fargate in private subnets: chat API and run workers
API contract	Version 1.24.0. `GET /api/contract` returns the version and the schema.
Auth	`cc_session` cookie (HttpOnly, Secure, SameSite=Strict, Path=/api). Cookie writes need `x-cc-csrf: 1`.
Access	The customer's roles and keys: the transcript role and key for chat state, the reader role and lake key for gold
PHI	Questions, answers, table previews and CSV files. They are kept only under the customer's prefix, never in logs or URLs.

Item	Value
Buckets	Customer data in the ClaimClarity bucket. The web app and the access logs have their own buckets.

Pages in this area

- Edge · Chat turn · State · Limits

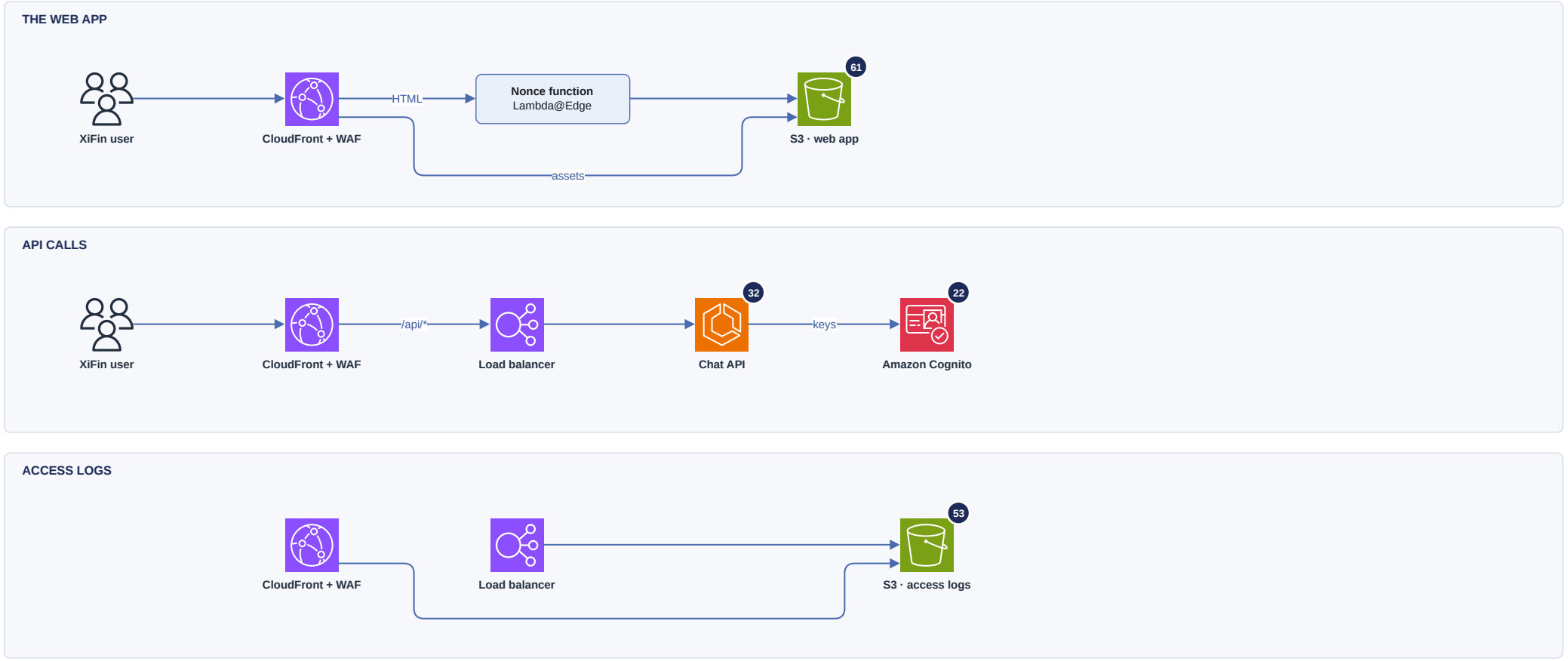
Related

Up: Overview · See also: Identity & access, Analyzer

Edge

One CloudFront distribution with AWS WAF serves the web app and the API on the same origin, so there is no CORS. The web app comes from its own S3 bucket, and API calls go through a load balancer to the chat API.

CLAIMCLARITY · EDGE



page with the detail

In this diagram

Item	What it does	Details
XiFin user	Opens the app and calls `/api/*` on the same origin	
CloudFront + WAF	TLS, WAF rules and rate limit, routing by path	CloudFront, WAF
Nonce function	Lambda@Edge. Returns `index.html` with a fresh CSP nonce on every response.	
S3 · web app	The web app bucket: hashed assets and the HTML template. No PHI.	Deploy
Load balancer	Application Load Balancer. Accepts traffic from CloudFront only.	ALB
Chat API	Checks the token on every call	Chat turn
Amazon Cognito	Publishes the keys that sign ID tokens	Token to credentials
S3 · access logs	The log bucket for CloudFront and load balancer access logs	Audit & logging

How it works

1. WAF checks every request at CloudFront.
2. HTML goes through the nonce function. `/assets/*` comes straight from the web app bucket.
3. `/api/*` goes to the load balancer with caching off, and the response is streamed.
4. The chat API checks the ID token's signature, audience, expiry and customer, using Cognito's keys.
5. CloudFront and the load balancer write access logs to the log bucket.

Specification

Item	Value
Caching	`/assets/*` cached for 1 year (immutable). HTML and `/api/*` are `no-store`.
Integrity	Subresource Integrity on every script and stylesheet. Edge features that rewrite JS, CSS or HTML stay off.

Item	Value
Load balancer access	Security group allows only the CloudFront prefix list. The listener also requires a secret origin header.
Timeouts	SSE heartbeat 15 s. CloudFront origin read timeout 60 s. ALB idle timeout 60 s.
Body limit	1 MiB at the API (413). WAF and the load balancer allow at least that.
Chat API egress	NAT gateway with Network Firewall, allowing only the Cognito hosts
Access logs	Full paths with opaque ids, so treated as PHI-bearing: encrypted, security team only, 1 year
Permissions	CloudFront reads the web app bucket through origin access control. The nonce function reads only the HTML template.
Failure	WAF block: 403, or 429 for the rate rule. Nonce function down: the app doesn't load, and the API keeps working.

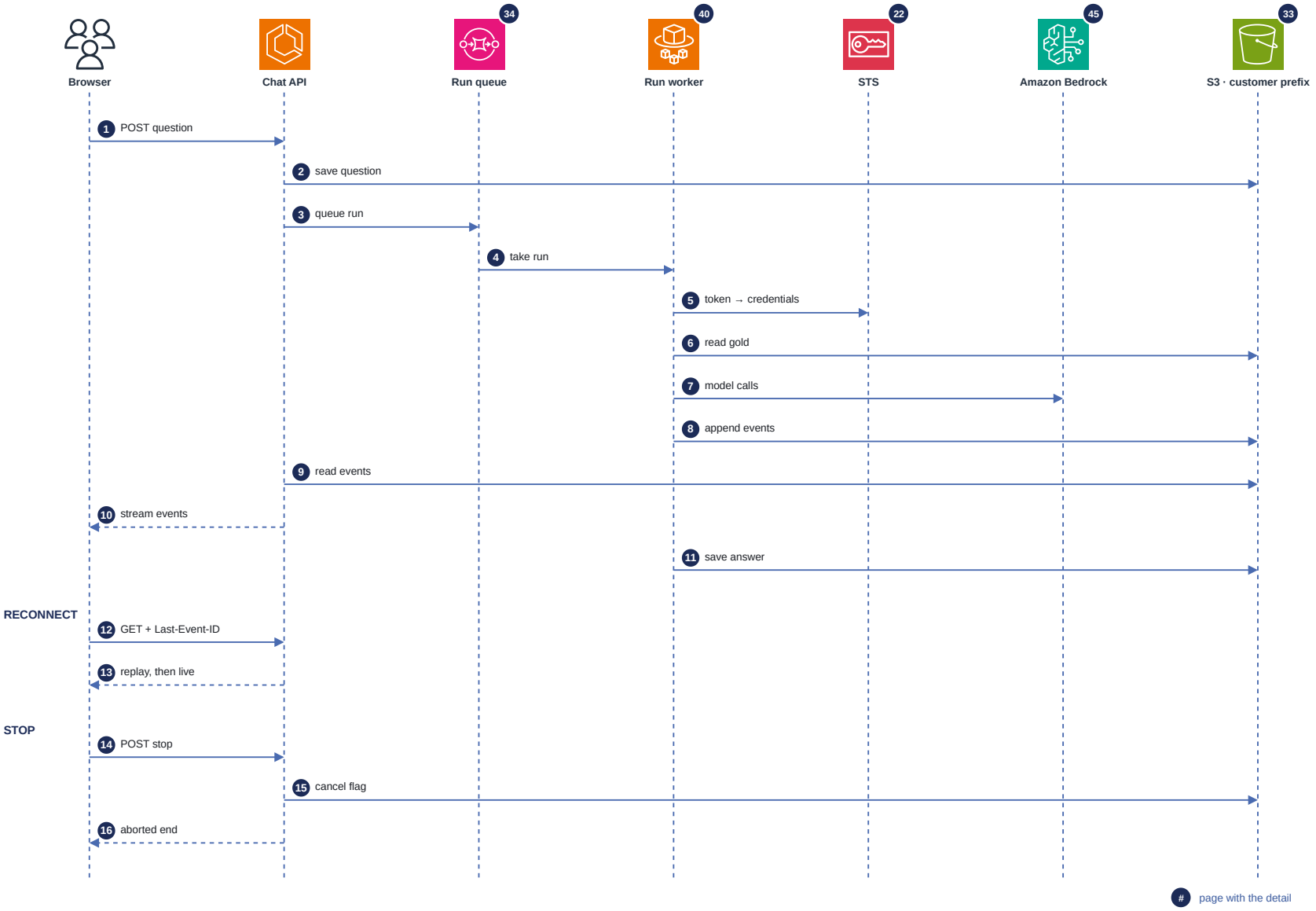
Related

Up: Chat backend · See also: Network, Deploy

Chat turn

One question goes from the browser to a run worker, and its events come back through S3 and the chat API. Because the events are stored, a dropped connection can resume on any API task, and Stop reaches the worker as a flag.

CLAIMCLARITY · CHAT TURN



In this diagram

Item	What it does	Details
Browser	Sends the question, reads the stream, reconnects, presses Stop	
Chat API	Saves the question, queues the run, relays the events	Limits
Run queue	Holds the run until a worker is free	Limits
Run worker	Runs the analyzer and writes events and the answer	Analyzer
STS	Exchanges the user's token for 15-minute reader credentials	Token to credentials
Amazon Bedrock	Claude Haiku	Model calls
S3 · customer prefix	The conversation, run-event chunks, cancel flag and gold	State

How it works

1. **Ask (1–3).** `POST /api/chat` with the cookie and `x-cc-csrf: 1`. The chat API saves the question with an If-Match write and queues the run. It answers `200 text/event-stream`.
2. **Run (4–8).** A worker takes the run, gets reader credentials from STS and reads gold. It calls the model, and appends numbered events to chunks: every 64 events, every 1 s, and at `message.end`.
3. **Stream (9–10).** The chat API reads new chunks every 0.5 s. It sends each event as `id: <messageId>:<seq>`, and a heartbeat after 15 s of silence.
4. **Save (11).** The worker adds the answer to the conversation. It never creates a conversation, so a deleted chat stays deleted.
5. **Reconnect (12–13).** `GET /api/chat/{messageId}/stream` with `Last-Event-ID`. The API replays events with a higher `seq`, then follows live. Replay is available for 60 s after the end.
6. **Stop (14–16).** `POST /api/chat/{messageId}/stop` writes a cancel flag. The worker checks it every 0.5 s, interrupts the SQL and ends with `message.end {finishReason: "aborted"}` within about 1 s.

Specification

Item	Value
Queue message	Ids (message, conversation, customer, user hash) and the user's ID token. Never the question text.
Reader credentials	15 minutes. Renewed with the same ID token when fewer than 3 minutes remain.
Outcomes	`stop`, `abstain`, `needs_input` (clarification, answerable for 30 minutes), `error`, `aborted`
Disconnect	No client attached for 10 s: the run is cancelled
Client retry	Reconnects after 250 ms, 1 s, 2 s and 4 s, then reloads the conversation. Also reconnects after 45 s of silence.
Errors	401 `auth_expired`, 404 `stream_gone`, 409 `clarification_expired`, 413, 422 `validation`, 429 `rate_limited`
PHI on the wire	Step events carry counts and column names only. Shown SQL has PHI literals replaced by `?`. No PHI in URLs.
Worker failure	Not retried. The stream stops, and after 60 s with no events the client reloads and the user retries.
Chunk write failure	Logged. The live stream goes on. Only a replay can miss those events.

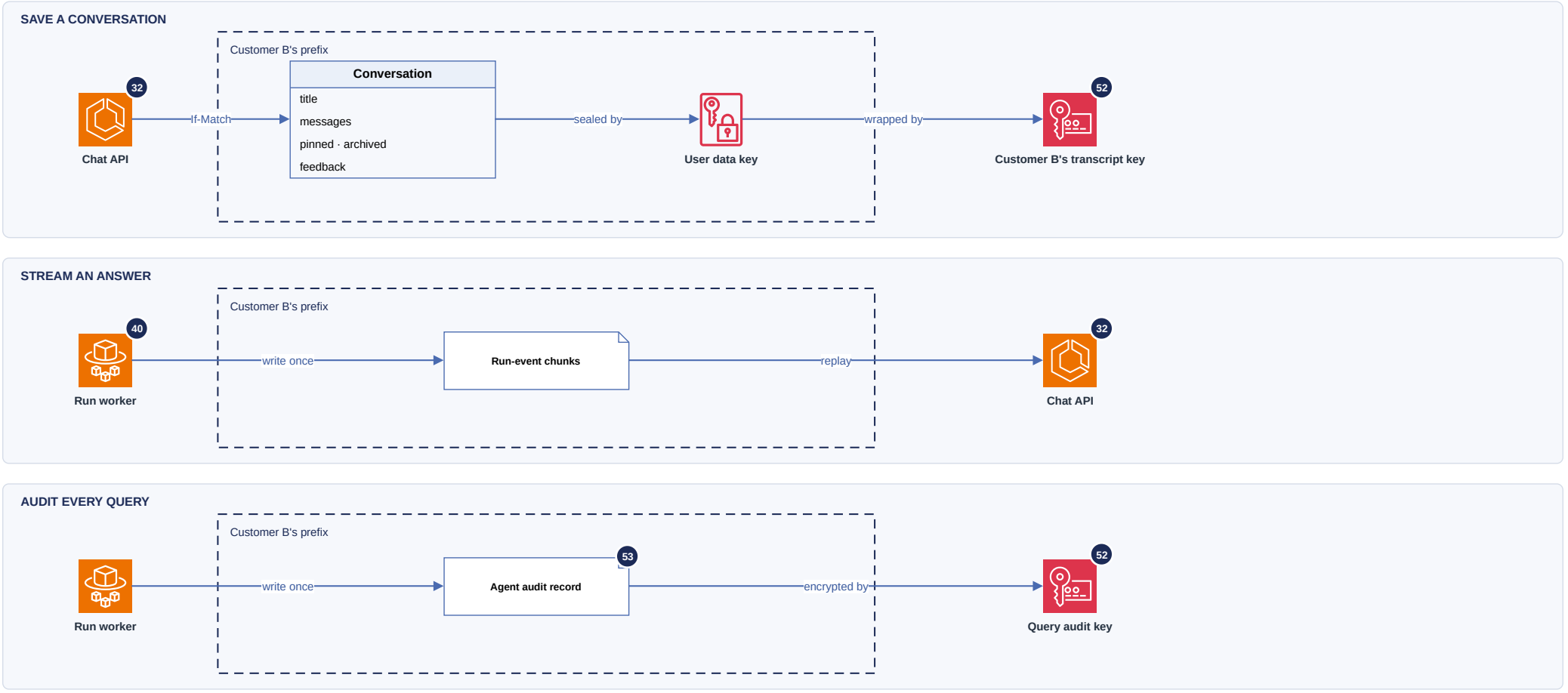
Related

Up: Chat backend · See also: State, Limits

State

All chat state is kept in S3 objects under the customer's prefix in the ClaimClarity bucket. There is no database. A conversation is one object, changed with an If-Match write, and its text is sealed with a per-user data key.

CLAIMCLARITY · CHAT STATE



In this diagram

Item	What it does	Details
Chat API	Reads and changes conversations	Chat turn
Conversation	One sealed object per chat: title, messages, pinned and archived, feedback	
User data key	A per-user AES-256 key. It seals the conversations and event chunks, and is stored only in wrapped form.	
Customer B's transcript key	KMS key that wraps every user data key	Encryption & keys
Run worker	Writes event chunks, the answer and the audit	Analyzer
Run-event chunks	Numbered write-once objects. Any API task can replay from them.	Chat turn
Agent audit record	One write-once record per SQL statement	Audit & logging
Query audit key	The platform key for audit records, with one statement per customer	Encryption & keys

How it works

1. To change a conversation, the chat API reads the object and its ETag, changes it, and writes it back with `If-Match`. A new conversation is written with `If-None-Match: *`.
2. If another task wrote first, S3 answers 412. The API reads again and retries, up to 8 times with jittered backoff.
3. The run worker writes event chunks once each (`If-None-Match: *`). It appends the answer with the same If-Match loop.
4. Every seal binds customer, user and object name, so a copied blob does not open elsewhere.
5. The run worker writes one agent audit record per SQL statement, through Customer B's audit writer role.

Specification

Objects (under `B/transcripts/users/{user}/` unless noted)

Object	Key	PHI
Conversation index	`meta/{conversation}.json` : ids, times, counts	No

Object	Key	PHI
Conversation	`content/{conversation}.bin`	Yes
Run-event chunk	`runs/{conversation}/{message}/{000000}.bin`	Yes
Cancel flag	`runs/{conversation}/{message}/cancel`	No
User data key	`dek.bin`, wrapped	No
CSV download	`downloads/{fileId}.bin`, 15 minutes	Yes
Conversation owner	`B/transcripts/conversation-owners/{conversation}.json`	No
Agent audit	`B/audit/queries/{date}/{request}.json`: ids, outcome, row count and the first 500 characters of the SQL	Yes

`{user}` is a 32-character hash of the customer and user ids.

Access, keys and retention

Item	Value
Chat API and run worker	Customer B's transcript role, session-tagged with the user: only that user's prefix
Audit	Customer B's audit writer role can only put records. The audit reader role can only read them.
Keys	Bucket SSE-KMS with Customer B's transcript key. Application seal: AES-256-GCM with the user data key. Audit records use the query audit key.
Retention	Conversations 90 days after the last change (users can choose less). Swept hourly. Agent audit 6 years, Object Lock, never deleted.
Delete	Deleting all chats removes the user's prefix, data key included. Disabling Customer B's transcript key makes every chat unreadable (410).
Temporary chats	Never saved. Held in one chat API task's memory for 1 hour idle, at most 20 per user. Their event chunks are deleted 60 s after the run.
Failure	Still conflicting after 8 tries: the request fails ("This conversation changed"). Nothing here logs text.

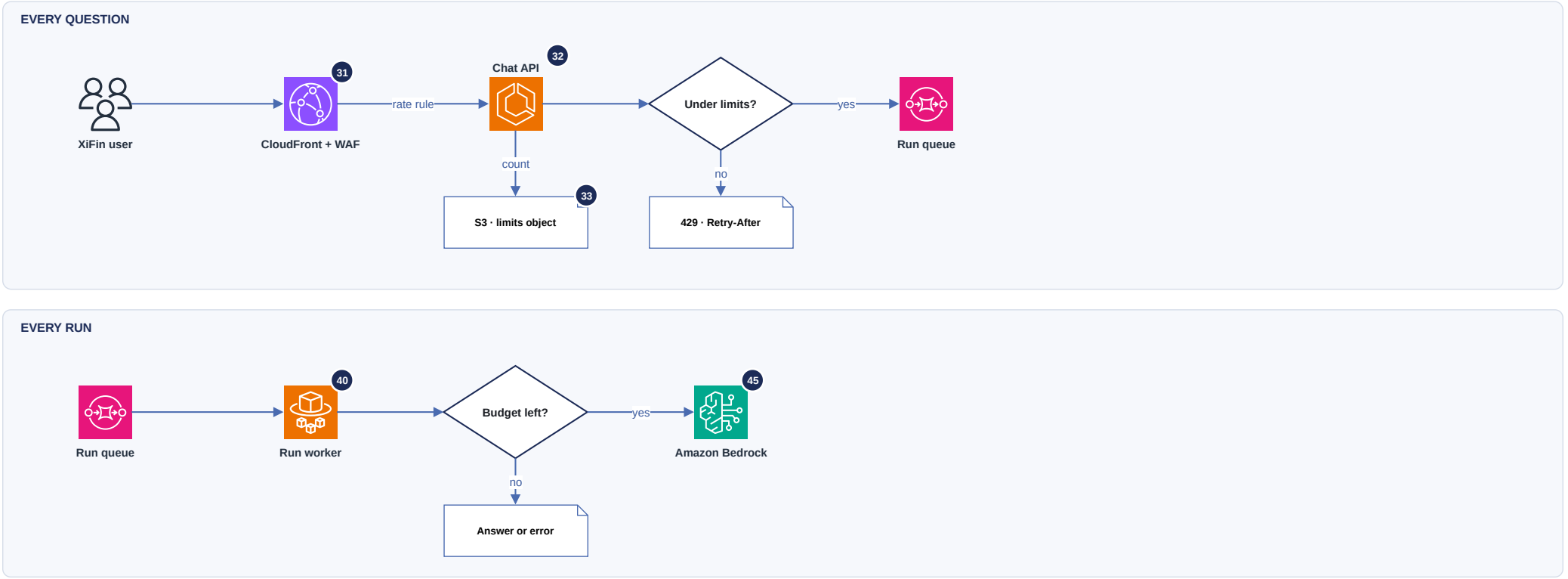
Related

Up: Chat backend · See also: S3 layout, Customer isolation

Limits

Limits are checked twice: once when a question arrives, and again while its run works. Every limit ends in a clear message or a normal answer outcome, never a spinner that doesn't stop.

CLAIMCLARITY · LIMITS



page with the detail

In this diagram

Item	What it does	Details
XiFin user	Waits out `Retry-After`, then retries up to 2 times	
CloudFront + WAF	Rate rule per client address	Edge
Chat API	Checks the body size and the run limits	Chat turn
S3 · limits object	Per customer: runs in flight, tokens used today, kill switch	State
Under limits?	Admits the run, or answers 429	
Run queue	Holds admitted runs only	
Run worker	Applies the budgets of one question	Analyzer
Budget left?	Lets the analyzer go on, or ends the run with an answer or an error	
Amazon Bedrock	Model calls while budget is left	Model calls

How it works

1. WAF applies its rate rule per client address.
2. The chat API rejects bodies over 1 MiB and bad requests.
3. It counts the user's unfinished runs and adds a lease to the customer's limits object with an If-Match write.
4. Over a limit, the answer is 429 with `Retry-After`. Otherwise the run is queued.
5. The run worker enforces the per-question budgets below.
6. At the end the worker releases the lease and adds the run's tokens to today's count.

Specification

Limit	Value	What the user sees
WAF rate rule	Per client address, 5-minute window	429, then retries
Request body	1 MiB; message text 8,000 characters	"Message too long" (413) or 422
Open runs per user	3	"Too many answers in progress", `Retry-After: 3`
Runs in flight per customer	Setting in the limits object	429, `Retry-After: 3`
Daily tokens per customer	Setting in the limits object	429 until midnight UTC
Kill switch per customer	Setting in the limits object	503 "Analyst unavailable"
Commands per question	12	The analyzer answers with what it has, or abstains
Model turns per question	30	`error`: "The analysis failed. Try again."
Output per model call	4,096 tokens	
SQL time	25 s per statement, 60 s per question	Step error `timeout`, or the analyzer stops querying
Rows	1,000 per query; 50-row preview plus CSV	"Showing n of N rows"
Run deadline	110 s (fits the 120 s ECS stop timeout)	`error`
Time in queue	2 minutes	`error`
Clarification	30 minutes	409 "Ask again"

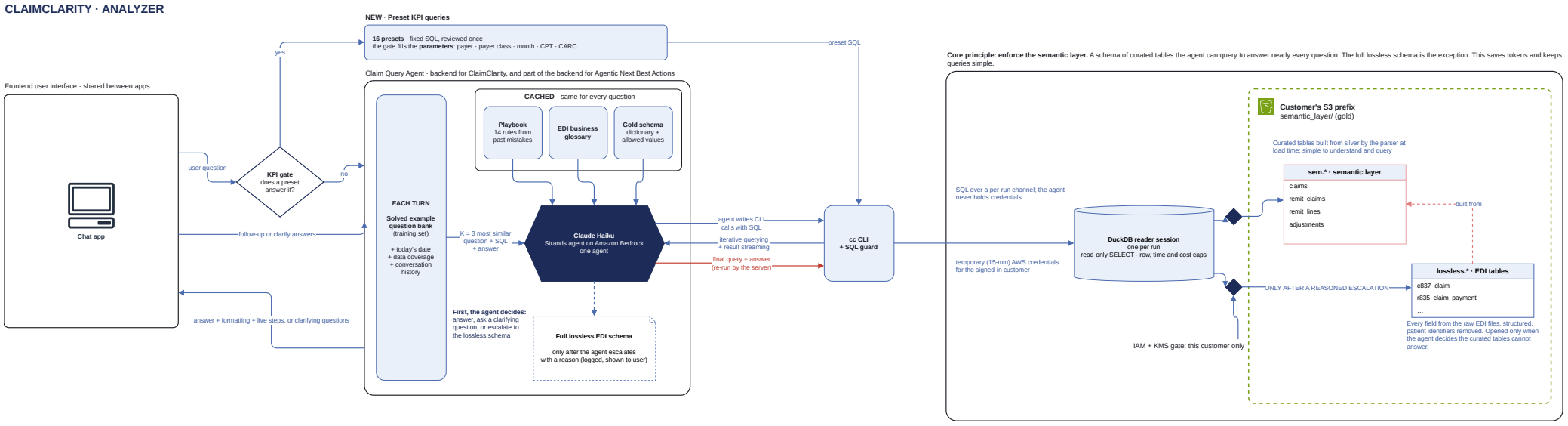
Item	Value
Lease expiry	The run deadline. A crashed worker never holds a slot for longer.
Who changes settings	The operations team, through the deploy pipeline only
Logging	Each refusal is an audit event with customer, user and reason

Related

Up: Chat backend · See also: Monitoring

Analyzer

The analyzer is code inside the run worker (ECS on Fargate) that turns one question into one answer, using Claude Haiku on Amazon Bedrock. A KPI gate sends each question to a preset KPI query or to the Claim Query Agent, and both read the customer's gold tables only through the cc CLI and SQL guard, on a read-only reader session.



In this diagram

Item	What it does (one sentence)	Details
Chat app	The frontend, shared between apps, where the user asks and sees answers, live steps and clarifying questions.	Chat turn
KPI gate	Decides whether one preset KPI query answers the question exactly.	KPI gate

Item	What it does (one sentence)	Details
Preset KPI queries	16 reviewed, fixed SQL queries; the gate fills their parameters (payer, payer class, month, CPT, CARC).	Preset KPI queries
Claim Query Agent	The Strands agent that answers every other question; also part of the backend for Agentic Next Best Actions.	Claim Query Agent
Each turn: solved example question bank	Gives the agent the 3 most similar solved questions, with today's date, the data coverage and the conversation history.	Claim Query Agent
Cached: Playbook, EDI business glossary, Gold schema	The parts of the agent's prompt that are the same for every question, cached by Bedrock.	Model calls
Claude Haiku	The model behind the gate and the agent, on Amazon Bedrock.	Model calls
Full lossless EDI schema	Described to the agent only after it escalates with a reason, which is logged and shown to the user.	cc CLI + SQL guard
cc CLI + SQL guard	The only way to the data: it checks every query from both paths, and the server re-runs the agent's final query.	cc CLI + SQL guard
DuckDB reader session	One read-only session per run, with row, time and cost caps, using the customer's 15-minute credentials.	Token to credentials
Customer's S3 prefix	Holds `sem.*` (gold, read by default) and `lossless.*` (PHI-stripped EDI tables, only after escalation).	S3 layout
IAM + KMS gate	AWS allows the reader to read this customer's data only (gate 6).	Customer isolation

How it works

1. The user asks a question in the chat app. The KPI gate checks whether a preset answers it exactly.
2. **Yes:** the preset's fixed SQL runs with the gate's values, and a short reply is written from the result.

3. **No:** the Claim Query Agent gets the question with 3 similar solved examples and its cached references. It answers, asks a clarifying question, or escalates to the lossless schema with a stated reason.
4. The agent writes cc CLI calls with SQL. Each one passes the SQL guard and runs on the DuckDB reader session, and the results stream back to the agent.
5. The agent submits its final query and answer. The server re-runs that query, so the numbers come from the database.
6. The answer streams back to the chat app with its formatting, SQL and live steps, or as clarifying questions.

Specification

Item	Value
Where it runs	Inside the run worker (ECS on Fargate). The analyzer is not a separate service.
Model	Claude Haiku on Amazon Bedrock, called with the run worker's task role
Data read	The signed-in customer's `sem.*`. `lossless.*` only after a reasoned escalation. Never another customer, never silver objects.
Credentials	The customer's reader role, for 15 minutes. The agent never holds credentials; SQL goes to the reader over a per-run channel.
Writes	None. Every query is one read-only SELECT.
Guard limits	25 s per query, 60 s total query time per question, 1,000 rows per query, keyless joins over 100 million row pairs refused
PHI	Query results are treated as PHI. Identifier columns are redacted by the cc CLI before results go anywhere.
Failure	If the gate or a preset fails, the question goes to the Claim Query Agent. If the agent ends without an answer, the user gets a retryable error.
Logging	Every query, escalation and refusal is audited under the customer's audit key. Model token usage is recorded per run.

Pages in this area

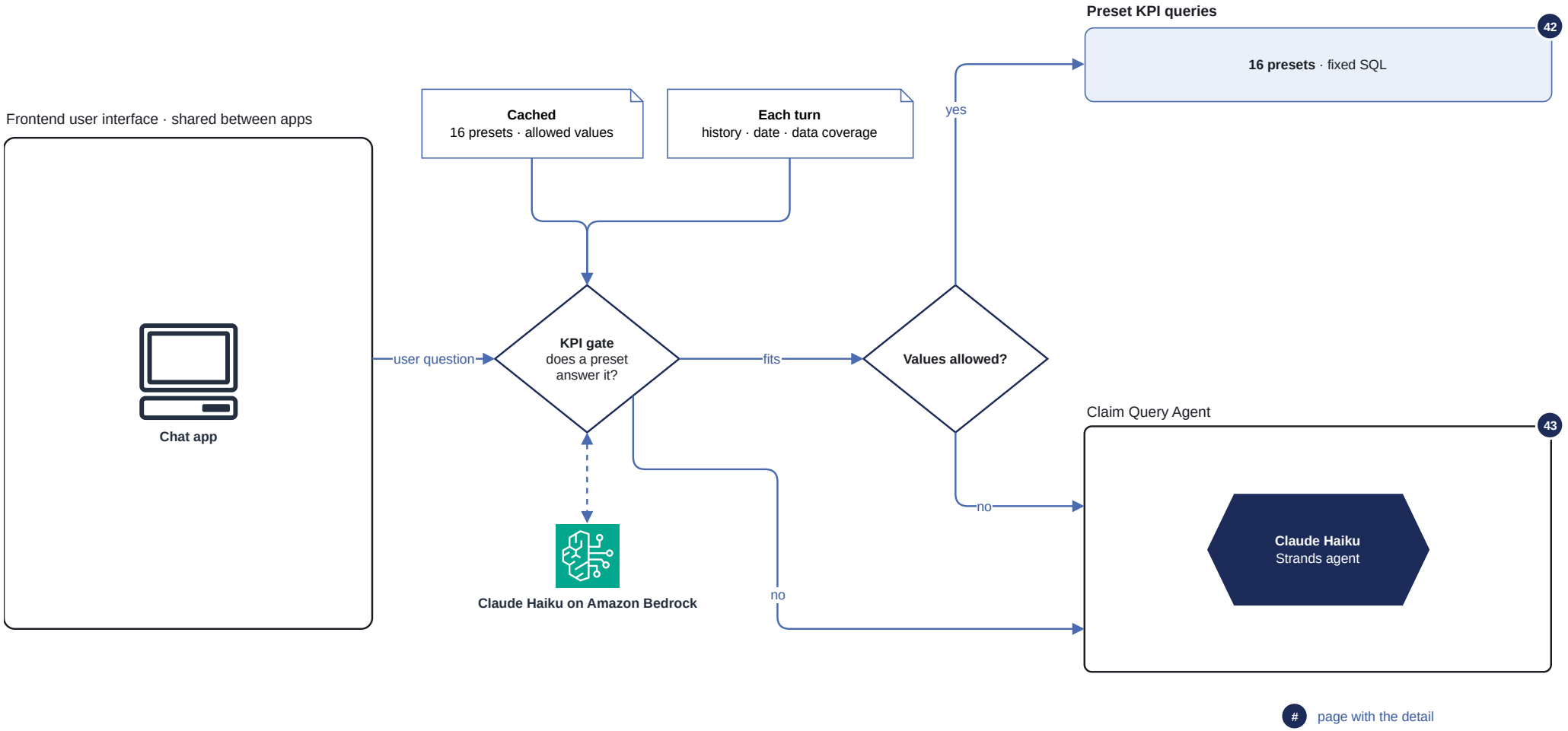
- KPI gate
- Preset KPI queries
- Claim Query Agent
- cc CLI + SQL guard
- Model calls

Related

Up: Overview · See also: Chat turn · Customer isolation

KPI gate

The KPI gate is the first step of every question: one call to Claude Haiku that decides whether one preset KPI query answers the question exactly and fills in its values. The server then checks the values, and anything that does not fit or fails goes to the Claim Query Agent.



In this diagram

Item	What it does (one sentence)	Details
Chat app	Sends the user's question.	Chat turn
Cached: 16 presets · allowed values	The gate's instructions, the preset catalog and the allowed values, cached for each customer and gold version.	Model calls
Each turn: history · date · data coverage	Earlier questions and how each was answered, today's date and the dates the data covers.	
KPI gate	Returns a decision: does a preset fit, which one, with which values and assumptions.	
Claude Haiku on Amazon Bedrock	Makes the decision; the gate has no tools.	Model calls
Values allowed?	Server code that accepts the decision only if the preset exists and every value is allowed.	
Preset KPI queries	Run when both checks say yes.	Preset KPI queries
Claim Query Agent	Answers when either check says no.	Claim Query Agent

How it works

1. The question arrives with the conversation history, today's date and the data coverage. History lets "and for Cigna?" keep the measure of the question before.
2. The gate asks Claude Haiku whether one preset computes exactly what was asked. Any extra filter, other grouping, comparison, "why", several numbers or different definition means no. When unsure, no. The gate never asks the user anything.
3. The server checks the decision: the preset exists, the values fill exactly its parameters, and every value is in its allowed list.
4. Yes: the preset runs. No, or any failure: the Claim Query Agent answers.
5. The user sees one step: "Standard KPI · <preset> · <values>" or "Custom analysis · <reason>".

Specification

Item	Value
Decision	`fits`, `template_id`, `values`, `assumptions`, `reason` (structured output)
Allowed values	Computed once per customer and gold version, through the SQL guard: payer names, payer classes, CARC codes, CPT codes, payment months, first-submission months
Preset catalog	`analyzer/kpi_templates.yaml`, version 1, 16 presets. Changes are reviewed.
Model	Claude Haiku on Amazon Bedrock, no tools, same session as the agent
Data in the prompt	Payer names and codes; the question and conversation history. No query rows.
Failure	Invalid values, an exception, a timeout, no structured output: the question goes to the Claim Query Agent and the failure is logged. The user never sees a gate error.
Logging	The run records the decision (preset id and values, or "custom analysis") and any failure type

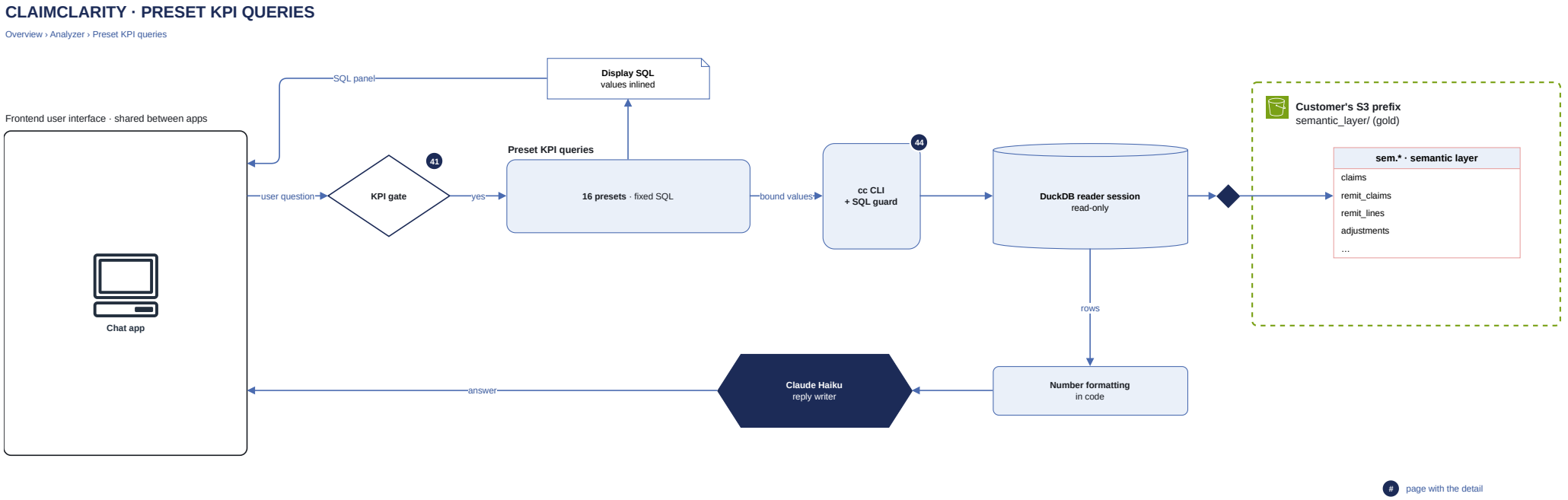
Related

Up: Analyzer · See also: Preset KPI queries · Model calls

Preset KPI queries

Overview › Analyzer › Preset KPI queries

When the KPI gate finds a fitting preset, the answer comes from reviewed, fixed SQL with the values bound as parameters, so the model cannot change the query. Code formats the numbers, and Claude Haiku writes a short reply from them.



In this diagram

Item	What it does (one sentence)	Details
Chat app	Sends the question and shows the answer and its SQL.	Chat turn
KPI gate	Picks the preset and its values.	KPI gate

Item	What it does (one sentence)	Details
Preset KPI queries	16 fixed SQL queries with named parameters for payer, payer class, month, CPT and CARC.	
Display SQL	The same SQL with the values written in, used wherever SQL is shown or saved.	
cc CLI + SQL guard	Checks and runs the query, exactly as for the agent.	cc CLI + SQL guard
DuckDB reader session	Runs the query read-only on the customer's gold.	cc CLI + SQL guard
Customer's S3 prefix · `sem.*`	The gold tables the presets read.	S3 layout
Number formatting	Formats rates as 12.3%, money as $1,234.56 and counts as 1,649, in code.	
Claude Haiku · reply writer	Writes 1–3 sentences from the formatted rows, copying the values exactly.	Model calls

How it works

1. The gate's preset and values become bound parameters. No value is pasted into the SQL that runs.
2. The query passes the SQL guard and runs on the reader session against `sem.*`.
3. Code formats the numbers in the result.
4. Claude Haiku writes a short reply with the gate's assumptions. It has no tools and writes no table.
5. The answer has the same parts as an agent answer (text, table, SQL, assumptions) and is marked `via="template:<id>@v1"`.
6. The display SQL is what the SQL panel, history, run events and exports show. It runs as-is to the same rows.

Specification

Item	Value
SQL that runs	The preset's fixed SQL with bound parameters. Never written by the model.
Data read	`sem.*` of the current gold version only. Presets never escalate.
Credentials	The customer's reader role, as for every query
Limits	The SQL guard's: 25 s per query, 60 s per question, 1,000 rows

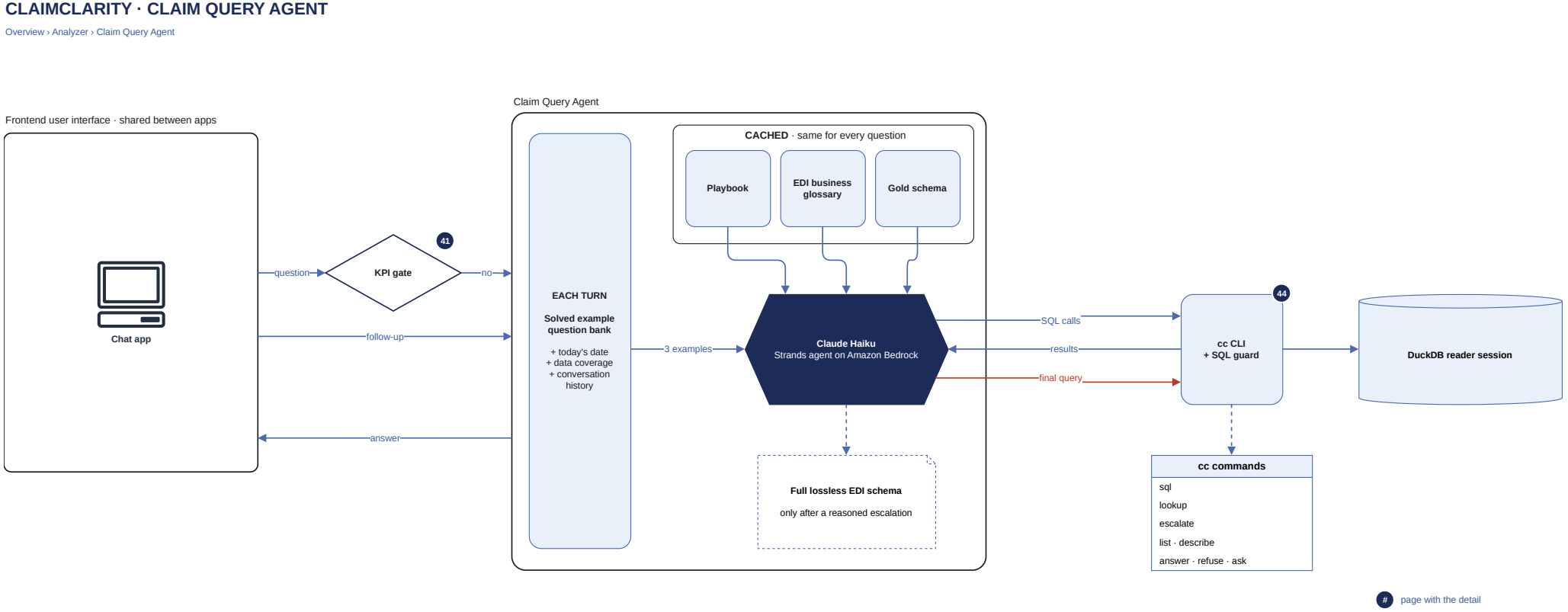
Item	Value
PHI	Result rows are treated as PHI; identifier columns are redacted
Failure	A SQL error, timeout or failed reply sends the question to the Claim Query Agent and is logged
Logging	Preset id, version and values, plus the query's audit record

Related

Up: Analyzer · See also: KPI gate · cc CLI + SQL guard

Claim Query Agent

The Claim Query Agent answers every question no preset fits: a Strands agent on Claude Haiku that writes its own read-only SQL over the customer's gold tables through the cc CLI. The server re-runs its final query, so the numbers in the answer come from the database, never from the model.



In this diagram

Item	What it does (one sentence)	Details
Chat app	Sends questions, follow-ups and clarify answers, and shows the answer.	Chat turn
KPI gate	Sends the question here when no preset fits.	KPI gate
Each turn: solved example question bank	Adds the 3 most similar solved questions, today's date, the data coverage and the conversation history.	
Cached: Playbook, EDI business glossary, Gold schema	Fixed guidance and the generated description of the gold tables, cached by Bedrock.	Model calls
Claude Haiku	The Strands agent on Amazon Bedrock; it decides to answer, ask or escalate.	Model calls
Full lossless EDI schema	Shown to the agent only after a reasoned escalation.	cc CLI + SQL guard
cc CLI + SQL guard	The agent's one tool: every command and query goes through it.	cc CLI + SQL guard
cc commands	`sql`, `lookup`, `escalate`, `list` and `describe`, then one of `answer`, `refuse` or `ask`.	
DuckDB reader session	Runs the queries read-only.	cc CLI + SQL guard

How it works

1. The agent gets the question, the cached references and the turn's examples and context.
2. It sends cc CLI calls with SQL; results stream back. Each command shows in the chat as a live step.
3. If gold cannot answer, it escalates with a reason and can then read the lossless schema.
4. It finishes with exactly one of: an answer (final SQL and text), a refusal, or up to 4 clarifying questions.
5. For an answer, the server re-runs the final SQL and builds the parts: text, SQL, table (50-row preview, full CSV download), chart when it fits, assumptions and caveats.
6. A clarify reply comes back as a follow-up and the agent answers without asking again.

Specification

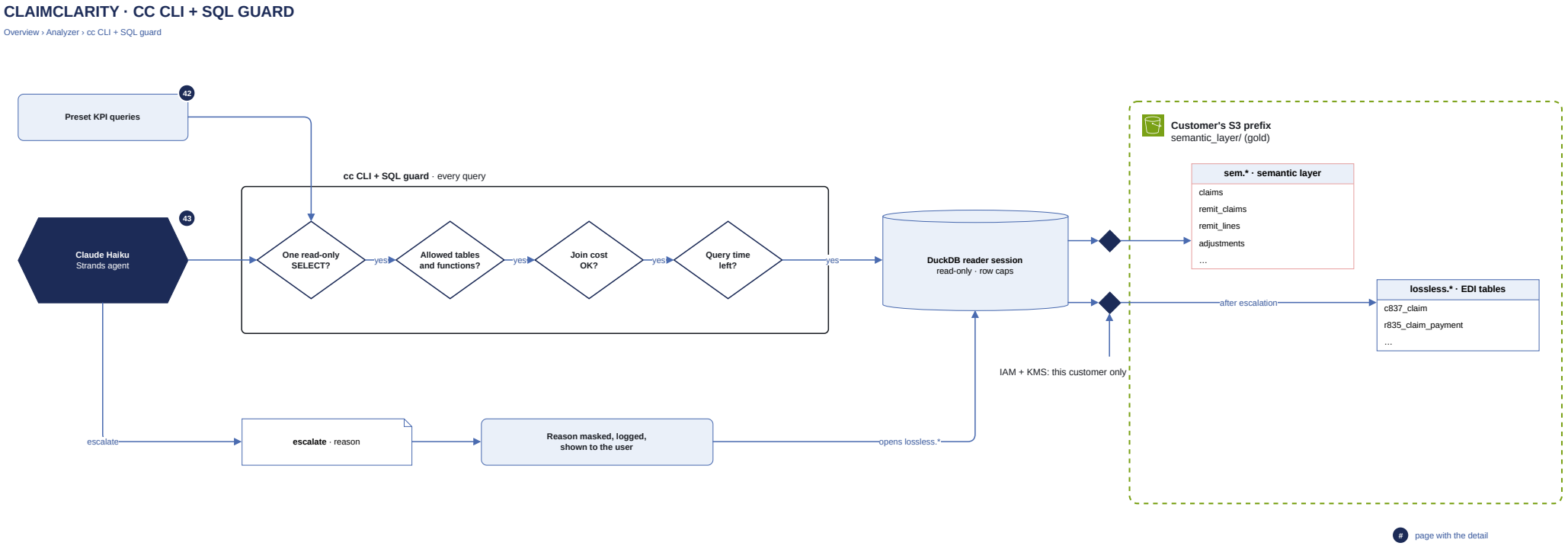
Item	Value
Model	Claude Haiku on Amazon Bedrock, through the Strands agent, one tool
Command budget	12 commands besides the final one; failed commands do not count
Query limits	The SQL guard's: 25 s per query, 60 s per question, 1,000 rows, 20 rows shown to the model
Customer scope	From the verified token only. The agent has no command that takes a customer.
Refuses	Patient identifiers, other customers, any change to data
Prompt safety	Query results are data, never instructions
Clarifications	Kept 30 minutes for the user to answer
Stop	The user's Stop cancels the model call; no further command runs
Failure	No final command: a retryable error ("I couldn't answer that")
Logging	A step event per command; an audit record per query, escalation and refusal; token usage per run

Related

Up: Analyzer · See also: cc CLI + SQL guard · Model calls

cc CLI + SQL guard

Every query from a preset or the agent passes the same checks before it reaches the reader session, which can see only one customer's data. The lossless tables open only when the agent states why gold cannot answer, and that reason is masked, logged and shown to the user.



In this diagram

Item	What it does (one sentence)	Details
Preset KPI queries	Send fixed SQL with bound values.	Preset KPI queries

Item	What it does (one sentence)	Details
Claude Haiku · Strands agent	Sends the agent's SQL, or an escalation.	Claim Query Agent
One read-only SELECT?	Refuses anything but a single SELECT.	
Allowed tables and functions?	Allows only `sem.*` (and `lossless.*` after escalation) and refuses functions that reach files, the network or settings.	
Join cost OK?	Refuses a join without a key over 100 million row pairs, before it runs.	
Query time left?	Stops a query at 25 s and the question at 60 s of total query time.	
DuckDB reader session	Runs the query read-only, caps the rows and redacts identifier columns.	Token to credentials
IAM + KMS gate	AWS allows the reader role to read only this customer's tables (gate 6).	Customer isolation
`sem.*` · semantic layer	The gold tables, read by default.	S3 layout
`lossless.*` · EDI tables	PHI-stripped EDI tables published next to gold, read only after escalation.	Gold publish
escalate · reason	The agent's request for lossless, with its reason.	
Reason masked, logged, shown to the user	Masks the reason, records it and re-opens the session with `lossless.*`.	

How it works

1. A query arrives from a preset or the agent.
2. The guard checks it is one read-only SELECT over allowed tables, with no banned functions.
3. It reads the query plan and refuses a keyless join that is too large, then runs the query with a time limit.
4. The reader session reads the customer's `sem.*`. Results come back capped, with identifier columns redacted.
5. To escalate, the agent sends a reason. It is masked, logged and shown as a step, and the session is re-opened for the same customer with `lossless.*` mounted. The guard then allows `lossless.*`.
6. Any refused check returns an error to the caller and nothing runs.

Specification

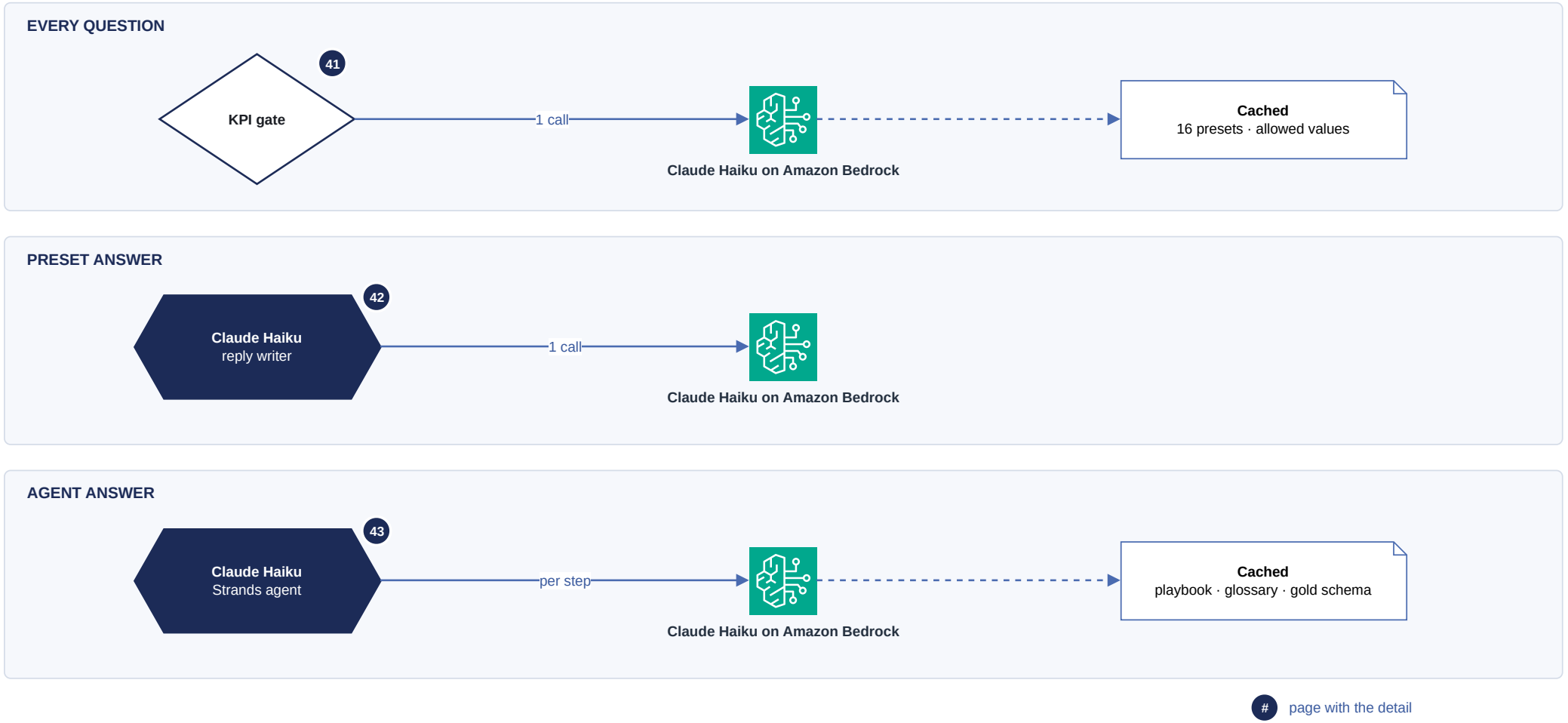
Item	Value
Statement	Exactly one; SELECT, UNION, INTERSECT or EXCEPT; no recursive CTEs; at most 32,768 characters
Tables	Schema-qualified `sem.*`, or `lossless.*` after escalation; no table functions, no catalog names
Banned functions	File readers (`read_*`, `parquet_*`, `glob` …), database scanners, `http_*`, secrets and settings, extension loading, `query`, `eval`, `shell`, `system`
Join cost	Keyless joins over 100,000,000 row pairs refused before they run
Time	25 s per query; 60 s total query time per question
Rows	1,000 per query (up to 10,000 if configured); 20 shown to the model; 50 in the answer preview, full result as a CSV download
Redaction	Patient and member names and ids, birth dates, addresses, SSNs, bank accounts, raw EDI text and file names
Reader session	One DuckDB session per run with the customer's reader role for 15 minutes; it holds only the mounted tables, cannot read other files or the network, and its settings are locked
Escalation	Only the agent can escalate, and only with a reason. The reason is put on one line, masked (quotes, dates, e-mails, ids, SSN- and phone-like values) and cut to 300 characters. Same customer only.
Keys	The customer's lake key for the tables; reads are allowed by the customer's reader role (gate 6)
Audit	Each query: customer, user, gold version, ok or failed, rows, seconds, error and the first 500 characters of the SQL, stored under the audit key. Escalations: requested, granted or failed. Server logs hold no SQL or rows.

Related

Up: Analyzer · See also: Customer isolation · Audit and logging

Model calls

Every model call in ClaimClarity goes from the run worker to Claude Haiku on Amazon Bedrock: one KPI gate call per question, then one reply-writer call for a preset answer or one call per step for an agent answer. The parts of the prompts that are the same for every question are cached by Bedrock.



In this diagram

Item	What it does (one sentence)	Details
KPI gate	One call per question.	KPI gate
Cached: 16 presets · allowed values	The gate's fixed prompt, cached per customer and gold version.	
Claude Haiku · reply writer	One call for a preset answer, no tools.	Preset KPI queries
Claude Haiku · Strands agent	One call per agent step, until its final command.	Claim Query Agent
Cached: playbook · glossary · gold schema	The agent's fixed prompt and tool, cached per customer and gold version.	
Claude Haiku on Amazon Bedrock	Runs every call.	Service catalog

How it works

1. Every question starts with the KPI gate call.
2. A preset answer adds one reply-writer call: two calls in all.
3. An agent answer adds one call per step. Each call returns one command; the run worker runs it and sends the result with the next call. The run ends at the final command, without another call.
4. Bedrock reads the cached part of each prompt from its prompt cache and processes only the new part.

Specification

Item	Value
Caller	The run worker's task role; it may invoke only Claude Haiku. The chat API and the parser cannot call Bedrock.
Calls per question	Preset answer: 2. Agent answer: 1 + one per step, at most 30 steps.
Typical time	Preset answer about 4 s; agent answer about 18 s
Output	At most 4,096 tokens per call

Item	Value
Prompt cache	Kept by Bedrock for 5 minutes after last use
Data sent	The question, the conversation history and query results (up to 20 rows per query, identifiers redacted). Treated as PHI.
Data kept by Bedrock	None; model invocation logging is off
Failure	A failed gate or reply call sends the question to the Claim Query Agent. A failed agent call gives a retryable error. Stop cancels the call.
Logging	Per run and model: input, output, cache-read and cache-write tokens, model time, calls. CloudTrail records the API calls without prompt content.

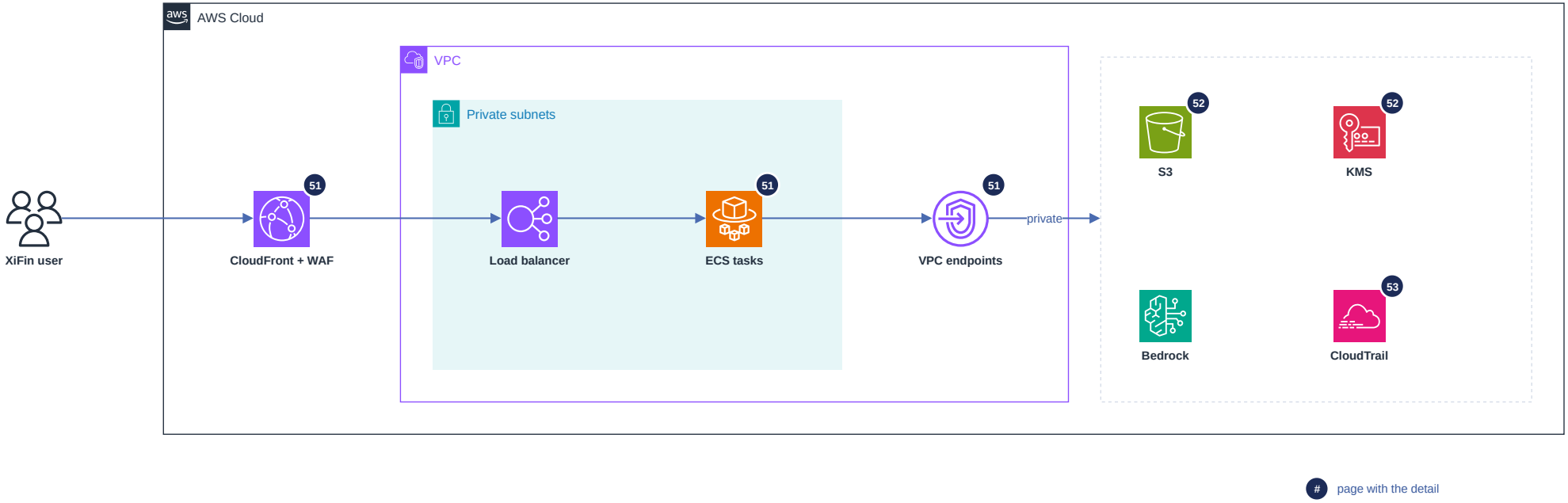
Related

Up: Analyzer · See also: Amazon Bedrock card · Monitoring

Network & security

ClaimClarity has one public entry point, CloudFront with AWS WAF, and every ECS task runs in private subnets that reach AWS only through VPC endpoints. Every object is encrypted with the customer's keys, and every AWS call and every analyzer SQL call is recorded.

CLAIMCLARITY · NETWORK & SECURITY



In this diagram

Item	What it does	Details
XiFin user	Reaches ClaimClarity over HTTPS only.	

Item	What it does	Details
CloudFront + WAF	The only public entry. Serves the web app and sends `/api/*` to the load balancer.	Network
Load balancer	Internal ALB in the private subnets. Accepts CloudFront traffic only.	Network
ECS tasks	Chat API, run workers and parser on Fargate, with no public IPs.	Network
VPC endpoints	Private access to S3 and the other AWS services.	Network
S3	The ClaimClarity bucket with all customer data. TLS only, SSE-KMS on every object.	Encryption keys
KMS	The customer's keys (lake, transcripts) and the platform query-audit key.	Encryption keys
Bedrock	Claude Haiku on Amazon Bedrock, reached through its endpoint.	Model calls
CloudTrail	Records every AWS call, allowed or denied.	Audit logging

How it works

1. The browser connects to CloudFront over HTTPS. WAF rules and rate limits run first.
2. CloudFront sends `/api/*` to the internal load balancer through a VPC origin.
3. The load balancer forwards to the chat API. Run workers and the parser accept no inbound traffic.
4. Tasks call S3, STS, KMS, Bedrock and the other services through VPC endpoints. The only internet path is the chat API's filtered egress to Amazon Cognito.
5. S3 encrypts every object with the writer's customer key. The bucket refuses requests without TLS.
6. CloudTrail, the agent audit and PHI-redacted application logs record what happened.

Specification

Control	Design
Public entry	CloudFront + WAF only. No public load balancer, no public IPs on tasks
Placement	All ECS tasks on Fargate in private subnets in two Availability Zones
AWS access	S3 gateway endpoint; interface endpoints for STS, KMS, Bedrock runtime, ECR api and dkr, CloudWatch Logs, SQS, Secrets Manager

Control	Design
Internet egress	Chat API only, through NAT + Network Firewall, to the Cognito hosts only
Buckets	One bucket holds all customer data. The web app and the access logs have their own buckets
In transit	TLS 1.2 or later on every hop. The bucket denies non-TLS requests
At rest	SSE-KMS on every object with the customer's keys; KMS on logs, queues and secrets
Audit	CloudTrail management and S3 data events; one agent-audit record per SQL call; redacted application logs
Retention	Raw 7 years · query audit 6 years · CloudTrail 6 years · network and app logs 1 year · transcripts 90 days

Pages in this area

- Network: subnets, security groups, VPC endpoints and the single egress
- Encryption keys: the customer's keys, key policies and who may decrypt what
- Audit logging: CloudTrail, the agent audit, redacted logs and retention

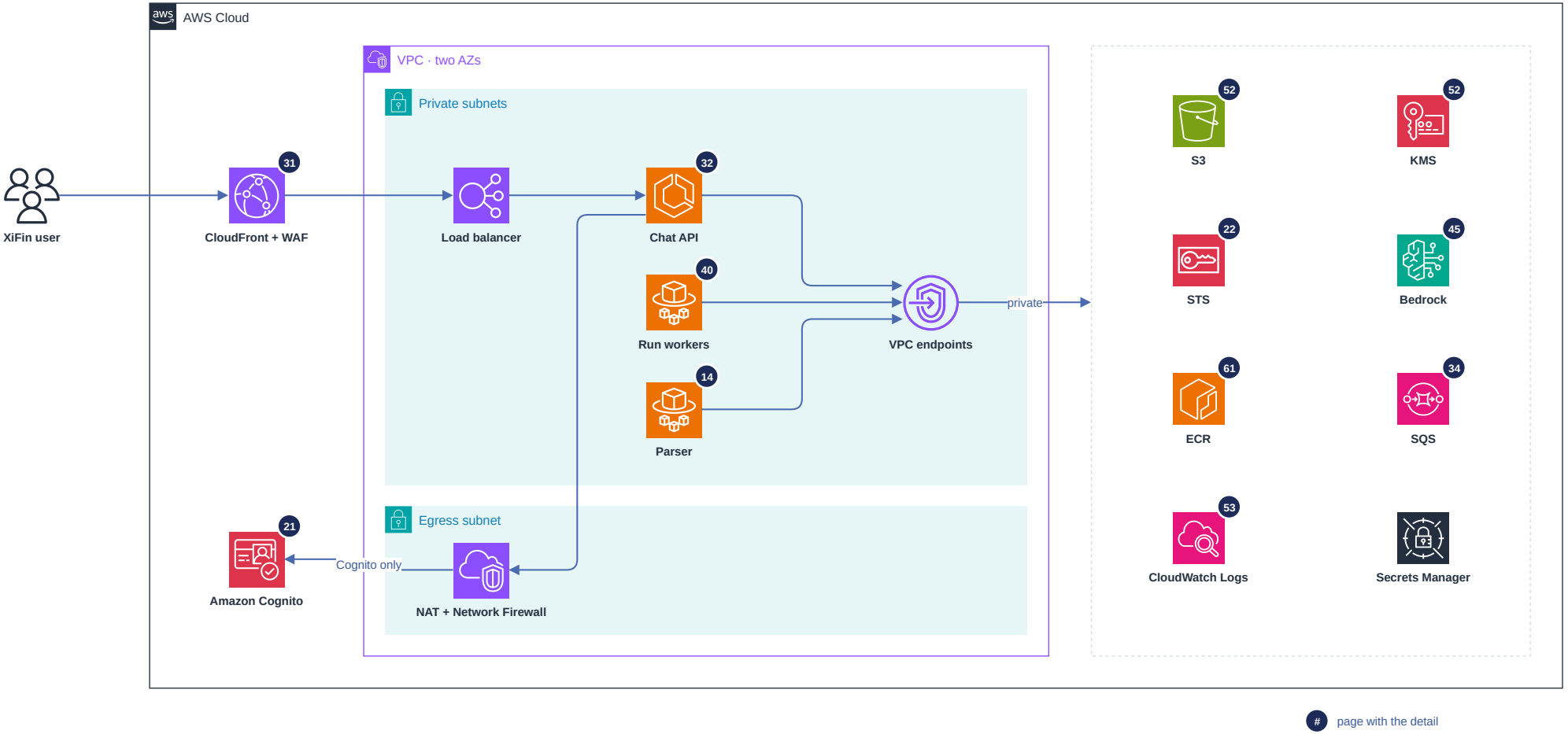
Related

Up: Overview · See also: Identity & access · Edge · IAM and KMS matrix

Network

The ClaimClarity VPC has no internet-facing resources: users enter through CloudFront and WAF, and tasks reach AWS through VPC endpoints. One filtered egress path lets the chat API reach Amazon Cognito and nothing else.

CLAIMCLARITY · NETWORK



In this diagram

Item	What it does	Details
XiFin user	Connects over HTTPS only.	
CloudFront + WAF	Public entry. Sends `/api/*` to the load balancer through a VPC origin.	Edge
Load balancer	Internal ALB. Its security group admits CloudFront's origin-facing addresses only.	
Chat API	Receives API calls from the load balancer.	Chat turn
Run workers	Pull runs from SQS. No inbound traffic.	Analyzer
Parser	Started by EventBridge. No inbound traffic.	Parser task
VPC endpoints	S3 gateway endpoint and eight interface endpoints, private DNS on.	
NAT + Network Firewall	The only egress. Allows HTTPS to the Cognito hosts only.	
Amazon Cognito	Signing keys and the sign-in token endpoint.	Sign-in
S3, KMS, STS, Bedrock, ECR, SQS, CloudWatch Logs, Secrets Manager	AWS services reached through the endpoints.	Encryption keys

How it works

1. CloudFront forwards `/api/*` to the internal load balancer through a CloudFront VPC origin.
2. The load balancer forwards HTTPS to the chat API.
3. All three task tiers call AWS through the VPC endpoints. S3 traffic, including ECR image layers, uses the gateway endpoint.
4. The chat API fetches the Cognito signing keys through NAT + Network Firewall at start, caches them for 24 hours, and refetches only for an unknown key id (at most every 5 minutes). The sign-in callback reaches the Cognito token endpoint the same way.
5. STS verifies tokens on the AWS side and needs no egress from the VPC.

Specification

Subnets

Subnet	Holds	Routes
Private (one per AZ, two AZs)	Load balancer, chat API, run workers, parser, interface endpoints	Default route to the NAT; S3 prefix list to the gateway endpoint
Egress (one per AZ)	NAT gateway, Network Firewall endpoint	Internet gateway

No task subnet routes to the internet gateway. Every ECS service and task definition sets `assignPublicIp = DISABLED`.

Security groups

Group	Inbound	Outbound
Load balancer	443 from the CloudFront origin-facing prefix list	App port to the chat API group
Chat API	App port from the load balancer group	443 to the endpoint group and the S3 prefix list; 443 to the internet, filtered to the Cognito hosts
Run workers	none	443 to the endpoint group and the S3 prefix list
Parser	none	443 to the endpoint group and the S3 prefix list
Endpoints	443 from the chat API, run worker and parser groups	none

VPC endpoints

Endpoint	Type	Endpoint policy
S3	Gateway	The ClaimClarity bucket and the ECR layer bucket
STS	Interface	AssumeRoleWithWebIdentity and AssumeRole for ClaimClarity roles
KMS	Interface	ClaimClarity keys
Bedrock runtime	Interface	InvokeModel on Claude Haiku only

Endpoint	Type	Endpoint policy
ECR api, ECR dkr	Interface	ClaimClarity repositories
CloudWatch Logs	Interface	ClaimClarity log groups
SQS	Interface	The run queue
Secrets Manager	Interface	ClaimClarity secrets

Egress filter

Allowed host	Used for
The Cognito user pool's issuer host	Signing keys (`/.well-known/jwks.json`)
The Cognito sign-in domain	Token endpoint at sign-in

Everything else is dropped and logged. If egress fails, signed-in users keep working on the cached keys and new sign-ins fail until it is back.

Logging

VPC flow logs, Network Firewall logs, load balancer access logs and WAF logs are kept 1 year, encrypted with KMS.

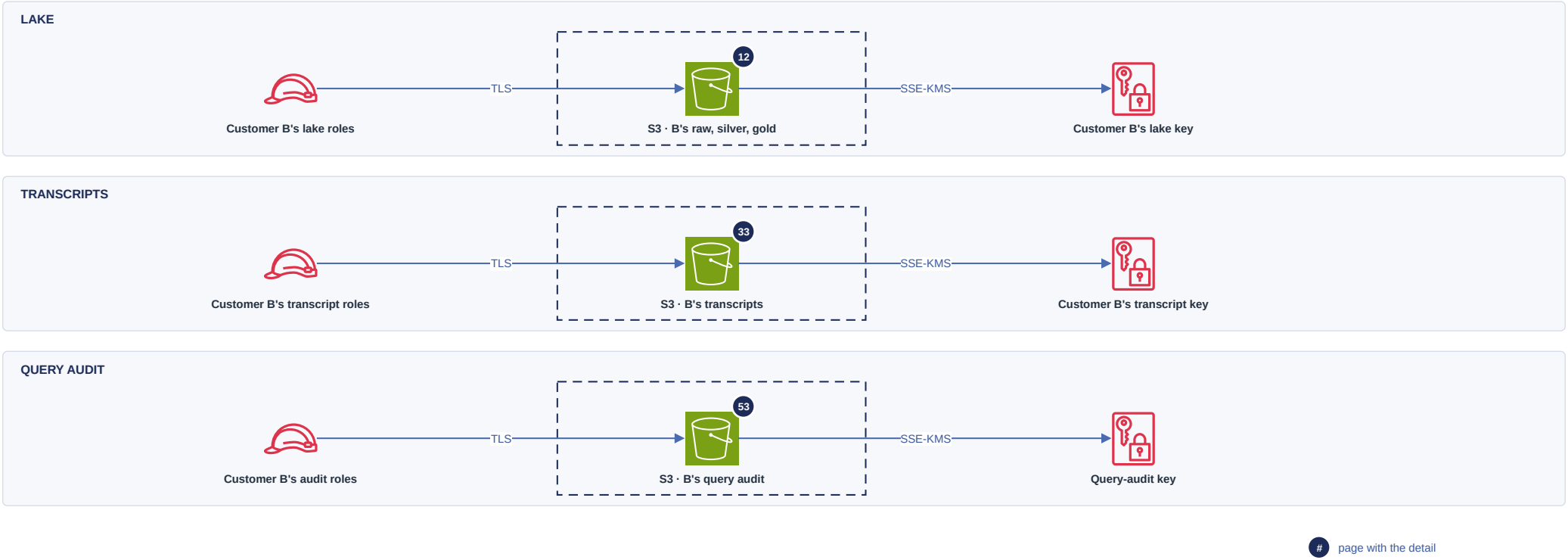
Related

Up: Network & security · See also: Edge · Encryption keys · VPC card

Encryption keys

Every object in the ClaimClarity bucket is encrypted with SSE-KMS under a key chosen by whose data it is and what kind it is: each customer has a lake key and a transcript key, and the query audit has one platform key with one statement per customer. Key policies are authoritative, so not even an IAM administrator can decrypt a customer's objects or rewrite the policies that protect them.

CLAIMCLARITY · ENCRYPTION KEYS



In this diagram

Item	What it does	Details
Customer B's lake roles	Parser, reader and SFTP copy roles, tagged with Customer B and B's lake key.	IAM and KMS matrix
S3 · B's raw, silver, gold	`B/raw/`, `B/parquet/`, `B/semantic_layer/`, plus staging and quarantine.	S3 layout
Customer B's lake key	Decrypts only for B's lake roles, only through S3, only under `B/`.	
Customer B's transcript roles	Transcript user, customer admin and eval roles.	State
S3 · B's transcripts	`B/transcripts/`: chats, metadata and de-identified exports.	State
Customer B's transcript key	A second key for B, for transcripts only. Lake roles cannot use it.	
Customer B's audit roles	Audit writer (assumed by the chat API) and audit reader (customer admins).	Audit logging
S3 · B's query audit	`B/audit/queries/`: write-once records, kept 6 years.	Audit logging
Query-audit key	One platform key with one statement per customer.	

How it works

1. Every request to the bucket must use TLS: the bucket policy denies `aws:SecureTransport = false`.
2. Every writing role carries a `kms_key_arn` tag. The bucket policy denies a PUT whose SSE-KMS key is not that key, or that names no key. The bucket default key is never used for customer objects.
3. S3 Bucket Keys are off, so each object's encryption context is its own ARN, which the key policy checks on every read and write.
4. Transcript message bodies are also encrypted by the chat API with a per-user data key from the transcript key, before they reach S3.
5. Disabling a customer's keys makes all of that customer's objects unreadable at once. Disabling only the transcript key removes the chats and leaves the lake and the query audit.

Specification

Keys

Key	Scope	Encrypts	Rotation
Lake key	Per customer	Raw, silver, gold, staging, quarantine	Automatic yearly; on demand by key admins
Transcript key	Per customer	`transcripts/` objects and per-user data keys	Automatic yearly
Query-audit key	Platform, one statement per customer	`<customer>/audit/queries/`	Automatic yearly
Platform default key	Platform	Bucket default only, never valid for customer objects	Automatic yearly

Who may use each key

Role	Key	Operations	Condition
Parser, reader, SFTP copy	Their customer's lake key	Decrypt, GenerateDataKey	Through S3, objects under `<customer>/`
Transcript user	Their customer's transcript key	Decrypt, GenerateDataKey	Own folder through S3; own data key only
Customer admin	Their customer's transcript key	Decrypt, GenerateDataKey	All users' folders, exports and admin objects; any user data key of the customer
Eval	Their customer's transcript key	Decrypt	Through S3, `transcripts/exports/` only
Support break-glass	The ticket's customer transcript key	Decrypt	Users' folders and data keys, read only
Audit writer	Query-audit key	Decrypt, GenerateDataKey	Through S3, own customer's `audit/queries/`
Audit reader	Query-audit key	Decrypt	Through S3, own customer's `audit/queries/`
Deploy role (Terraform)	All	Key policy and lifecycle	No data use
Key admin	All	Enable, disable, rotate, schedule deletion	No decrypt, no grants, no policy changes

Key policies do not delegate to the account, so IAM policies cannot widen them. Per role: IAM and KMS matrix.

Failure behaviour

Case	Result
Key disabled	Every read and write under it fails until it is re-enabled
Key scheduled for deletion	Unusable during the 7–30 day waiting period, which can be cancelled; afterwards the data is gone
Wrong key on a PUT	Denied by the bucket policy
Wrong encryption context	KMS refuses

Logging

Every Decrypt and GenerateDataKey is a CloudTrail event with the role and the object ARN. Key policy changes, disables and deletion schedules raise alarms.

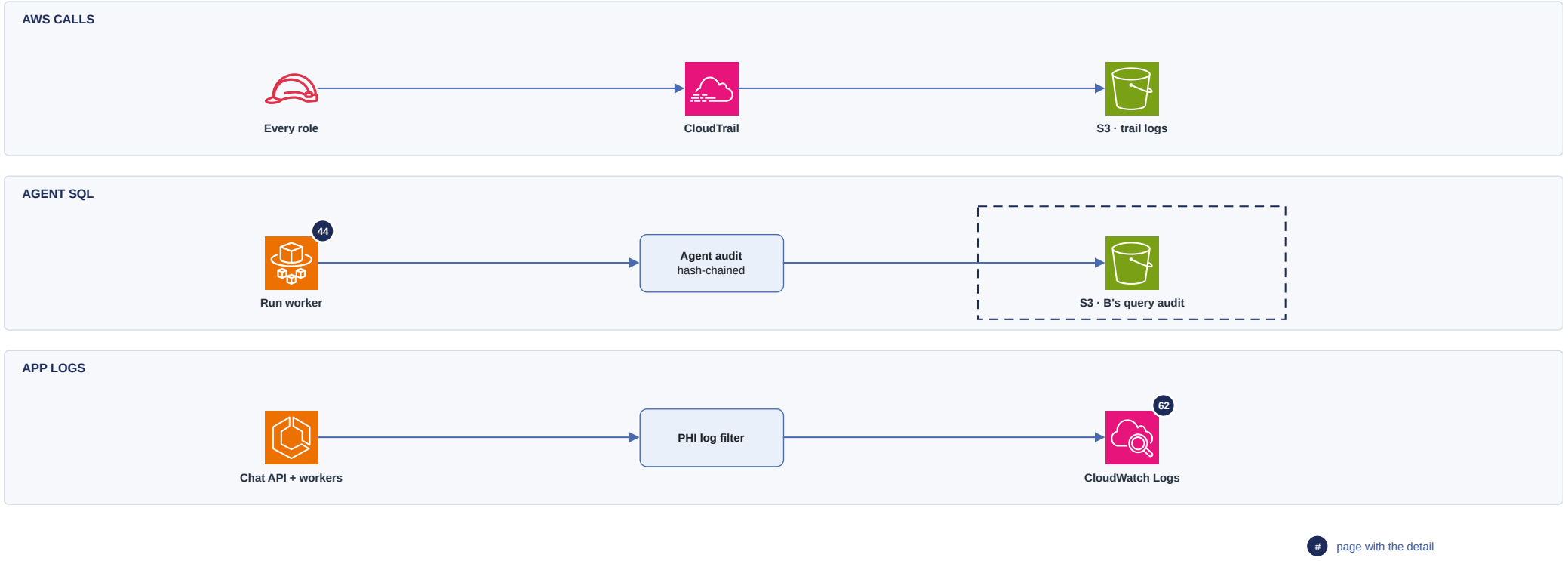
Related

Up: Network & security · See also: Customer isolation · S3 layout · IAM and KMS matrix

Audit logging

CloudTrail records which AWS identity touched which object or key, the agent audit records which SQL the analyzer ran for whom, and the application logs record what the service did. The first two are kept write-once for 6 years; the application logs are redacted so they carry no PHI.

CLAIMCLARITY · AUDIT LOGGING



In this diagram

Item	What it does	Details
Every role	Any task role, pipeline, IT's SFTP copy or person making an AWS call.	IAM and KMS matrix
CloudTrail	Records management events and S3 data events, allowed or denied.	CloudTrail card
S3 · trail logs	The log bucket: Object Lock, its own KMS key, 6 years.	
Run worker	Runs every analyzer SQL call through one entry point.	SQL guard & escalation
Agent audit	One hash-chained record per SQL call and per session opening.	
S3 · B's query audit	`B/audit/queries/<date>/<request id>.json`, write-once for 6 years.	Encryption keys
Chat API + workers	Write application and access logs.	
PHI log filter	Redacts every log record before it is written.	
CloudWatch Logs	KMS-encrypted log groups, kept 1 year.	Monitoring

How it works

1. CloudTrail records every management event (IAM, STS, KMS, ECS, Cognito) and S3 data events for the ClaimClarity bucket, including denials. It delivers to the log bucket with log file validation on.
2. Each analyzer SQL call writes one agent-audit record, whatever the outcome. Each record carries a sequence number, the previous record's hash and its own SHA-256 hash, so an edit, reorder or insert breaks the chain.
3. At the end of the request the chat API assumes Customer B's audit writer role and writes the request's records as one object. The writer can only add objects, and the bucket policy requires create-only writes, Object Lock COMPLIANCE and 6 years of retention.
4. Application logs pass the PHI filter: quoted text, emails, SSNs, phone numbers, dates and ids are masked, tracebacks lose their messages, paths become route shapes and client IPs are cut to /24 or /48.
5. Server logs never hold SQL, user text, row values or tokens.

Specification

Agent audit record

Field	Content
`kind`, `outcome`	Session open, SQL call or escalation grant; `ok`, `blocked`, `error` or `expired`
`customer`, `user`, `request_id`	From the verified token; the request id also appears in the STS session name
`version`, `escalate`	Gold version read; whether `lossless.*` was mounted, with the stated reason (masked, at most 300 characters)
`sql`, `sql_sha256`	The first 500 characters of the SQL, and its SHA-256
`row_count`, `column_count`	Result size. Result cells are never stored
`seq`, `prev_hash`, `hash`	The hash chain

The query audit can hold literal values the user typed, so it is treated as PHI and stored under the customer's prefix with the query-audit key.

Security alarms (on the CloudTrail stream)

Alarm	Fires on
Cross-customer denial	`AccessDenied` on S3 or KMS for a reader, parser or transcript role
Key change	PutKeyPolicy, DisableKey or ScheduleKeyDeletion on a ClaimClarity key
Bucket change	Bucket policy, lifecycle, versioning, Object Lock or encryption changes
Break-glass	Any use of the support break-glass role
Root use	Any sign-in or call by the root user

Retention

Record	Kept	Enforced by
Raw EDI files	7 years	Object Lock COMPLIANCE; deletes denied
Query audit	6 years	Object Lock COMPLIANCE; deletes denied
CloudTrail	6 years	Object Lock on the log bucket
Network and application logs	1 year	CloudWatch Logs retention
Transcripts	90 days	S3 lifecycle; users can delete sooner

Failure behaviour

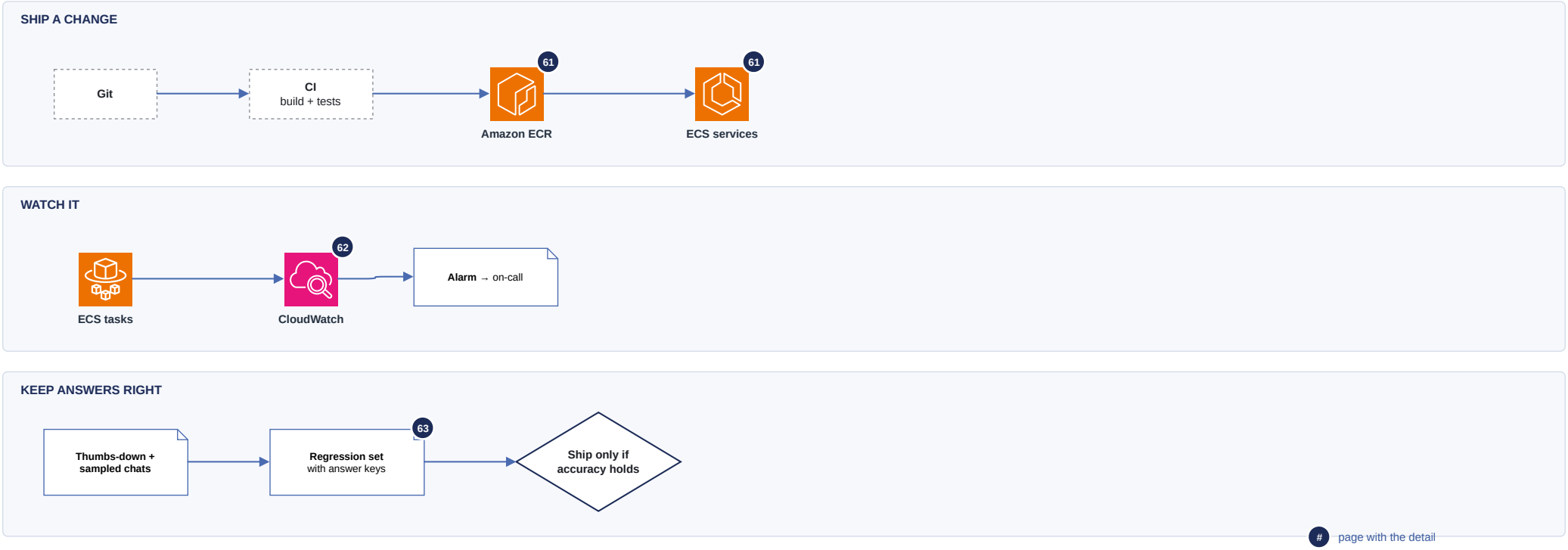
Case	Result
Agent-audit write fails	The SQL result is not returned. The run fails rather than run unaudited SQL
Query-audit object already exists	The write is refused. Records are never replaced
CloudTrail delivery delayed	AWS retries. Nothing is lost

Related

Up: Network & security · See also: Encryption keys · Monitoring · SQL guard & escalation

Operations

Every change ships through Git, CI and ECR to ECS, and analyzer changes must also keep their accuracy. CloudWatch watches the running tasks and pages on-call when an alarm fires.



In this diagram

Item	What it does	Details
Git	The repository. Changes come in as reviewed pull requests.	Deploy

Item	What it does	Details
CI	Builds and tests every change, including the eval gate for analyzer changes	Deploy
Amazon ECR	Holds the chat API, run worker and parser images	Deploy
ECS services	Run the new images on Fargate after the health checks pass	Deploy
ECS tasks	Write redacted logs and metrics	Monitoring
CloudWatch	Logs, metrics and alarms	Monitoring
Alarm → on-call	Notifies on-call	Monitoring
Thumbs-down + sampled chats	Become candidate test questions	Eval regression
Regression set	Questions with answer keys, re-run on every analyzer change	Eval regression
Ship only if accuracy holds	Blocks an analyzer change that is less accurate	Eval regression

How it works

1. A pull request runs CI. Analyzer changes also run the regression set.
2. On merge, CI pushes images to ECR, and the deploy step rolls the ECS services.
3. The tasks log to CloudWatch, and alarms notify on-call.
4. Thumbs-down answers and sampled chats keep the regression set growing.

Specification

Item	Value
Deploy identity	GitHub Actions assumes the deploy role through OIDC. No AWS keys are stored in GitHub.
Deploy role	Cannot read customer data, use the customer's keys, or change bucket or key policies
Logs	No question text, SQL or row values in any log. App logs are kept 1 year.

Pages in this area

- Deploy · Monitoring · Eval regression

Related

Up: Overview · See also: Chat backend, Audit & logging

Deploy

Three things ship from one repository: container images, the web app and the gold view SQL. Each goes through CI from a reviewed pull request and is deployed by a pipeline role, not by a person.

CLAIMCLARITY · DEPLOY



In this diagram

Item	What it does	Details
Git	The protected main branch	
CI	Lint, type checks, unit, contract and security tests, and the eval gate for analyzer changes	Eval regression
Amazon ECR	One repository per image: chat API, run worker, parser	ECR
ECS services	Rolling deploy of the chat API and run workers	ECS
Healthy?	New tasks must pass their health checks	
New tasks live / Roll back	Old tasks stop, or the circuit breaker rolls back	
S3 · web app	The web app bucket. New assets go in first, then the HTML template.	Edge
CloudFront	Invalidates the HTML paths only	Edge
Parser task	Its image carries the view SQL	Parser task
S3 · gold	The next parser run publishes the views with a new version	Gold publish

How it works

1. CI runs every check. A critical image scan finding, or a failed eval gate, stops the pipeline.
2. CI pushes images tagged with the commit SHA. Task definitions pin the digest.
3. The chat API rolls at 100–200 % with the `GET /api/health` check. Old tasks drain open streams for up to 120 s, and clients reconnect and replay.
4. Run workers stop taking new runs and finish the current one. The 110 s run deadline fits the 120 s stop timeout.
5. The web app's new hashed assets are uploaded without overwriting old ones, then the HTML template, then a CloudFront invalidation. The SRI, header and CSP checks then run against the live site.
6. A view SQL change ships in the parser image. CI checks every gold answer against its answer key first.

Specification

Item	Value
Deploy role can	Push to the three ECR repositories, register task definitions, update the ECS services, pass only the task and execution roles, write the web app bucket, invalidate CloudFront
Deploy role cannot	Read the ClaimClarity bucket, use the customer's keys, or change bucket and key policies (the infrastructure pipeline does that)
Images	Immutable tags. Scanned on push. The last 50 are kept for rollback.
Rollback	ECS circuit breaker, or redeploy the previous revision. Web app: restore the previous HTML template.
Production settings	Cookie auth, SQL literal masking `phi` or `all`, the paid-model switch on in production only

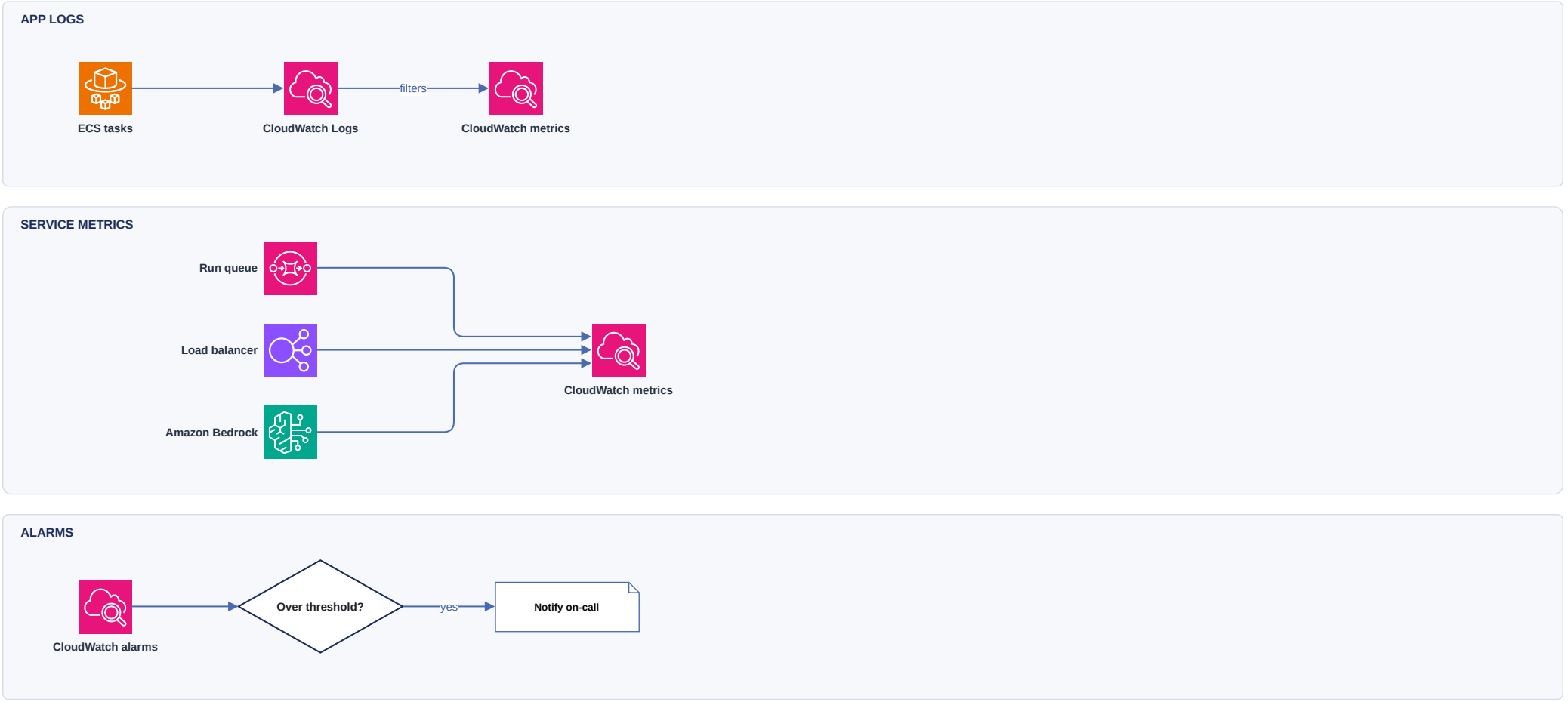
Related

Up: Operations · See also: Monitoring

Monitoring

Every task logs to CloudWatch, and the logs never hold question text, SQL or row values. Metrics from the logs and from AWS services drive alarms that notify on-call.

CLAIMCLARITY · MONITORING



In this diagram

Item	What it does	Details
ECS tasks	Chat API, run workers and parser. They write redacted logs, plus one metric record per answer and per batch.	
CloudWatch Logs	One encrypted log group per service	CloudWatch
CloudWatch metrics	Metrics from metric filters, embedded metric records and AWS services	
Run queue · Load balancer · Amazon Bedrock	Publish their own metrics	
CloudWatch alarms	Compare the metrics with the thresholds below	
Over threshold?	Fires the alarm	
Notify on-call	An SNS topic for e-mail and chat	

How it works

1. A redaction filter masks identifiers in log text. Exceptions are logged as type and line only.
2. Access logs record route shapes (`/api/conversations/:id`) and client addresses cut to /24 or /48.
3. Each answer writes one record with ids, counts and timings: outcome, tokens, estimated cost, time to first token, model time, SQL time.
4. Metric filters count rate-limit refusals, chunk write failures, write conflicts and data escalations.
5. Alarms notify on-call through SNS.

Specification

Alarm	Starting threshold	Action
API 5xx	Above 1 % of requests for 5 minutes	Page
Healthy chat API tasks	Below 2 for 2 minutes	Page

Alarm	Starting threshold	Action
Answer errors	Above 5 % of answers for 15 minutes	Page
Time to first token	p95 above 15 s for 15 minutes	Notify
Bedrock	p95 latency above 10 s, or any throttles	Notify
Daily tokens	A customer above 80 % of its budget	Notify
Run queue	Oldest message older than 30 s for 5 minutes	Page
Dead-letter queue	Any message	Notify
Parser	Any failed batch	Page
Quarantine	Any file quarantined	Notify the data team

Item	Value
Log groups	`/claimclarity/chat-api`, `/claimclarity/run-worker`, `/claimclarity/parser`: 1 year, KMS-encrypted with the logs key
Permissions	Each task writes only its own log group, and metrics only in the `ClaimClarity` namespace. The operations team reads them without any customer access.
Never logged	Question and answer text, SQL, row values, titles, feedback comments, tokens, cookies

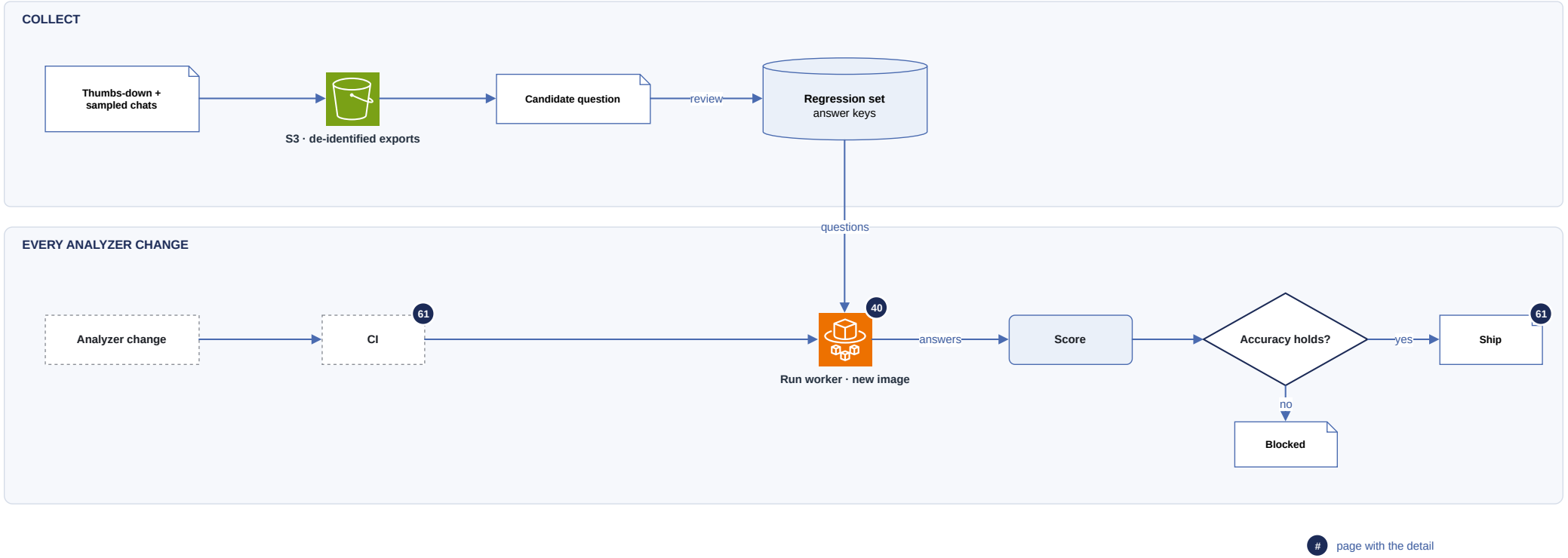
Related

Up: Operations · See also: Limits, Audit & logging

Eval regression

Real questions, and especially the thumbs-down ones, become test questions with independent answer keys. Every analyzer change is re-run on this set, and it ships only if accuracy does not drop.

CLAIMCLARITY · EVAL REGRESSION



In this diagram

Item	What it does	Details
Thumbs-down + sampled chats	Answers users marked wrong, plus a small sample of the rest	
S3 · de-identified exports	Under the customer's `transcripts/exports/` prefix, with patient-level values removed	State
Candidate question	Written by a reviewer, with gold SQL	
Regression set	Questions, answer keys and reworded variants, in train, validation and sealed test splits	
Analyzer change	A pull request to the analyzer	
CI	Starts the eval run	Deploy
Run worker · new image	The changed analyzer answers every question on the synthetic test data set	Analyzer
Score	Each answer is matched against its answer key	
Accuracy holds?	Compares the new version with the current one, question by question	
Ship / Blocked	Merge and deploy, or block	Deploy

How it works

1. Thumbs-down answers and a small sample of other answers are exported with patient-level values removed. The eval role can read only these exports.
2. A reviewer turns an export into a question with gold SQL. Its answer key is computed a second, independent way, and the two must agree.
3. Rewordings of a question share one answer key and always sit in the same split.
4. A change to the analyzer starts the run worker image from that change on the validation split, with at least two repeats.
5. The scorer matches columns by value and rows within tolerance. Safety questions pass only if the analyzer refuses.
6. The change ships only if accuracy and answered-and-wrong are no worse, every safety question is still refused, and the rewordings of each question still agree.

Specification

Item	Value
Data	The synthetic test data set only. The eval run never reads customer data.
Model	Claude Haiku on Amazon Bedrock, the same as production
Reported	Accuracy, right-in-both-repeats, answered-and-wrong, cost per correct answer
Comparison	Paired by question, with a 95 % bootstrap interval
Permissions	The eval role reads only `transcripts/exports/`. CI's eval role may call Bedrock and nothing else.
PHI	None in the regression set or in the CI results
Blocking	A failed gate blocks the merge. An override needs a written reason in the pull request.

Related

Up: Operations · See also: Analyzer, Deploy

AWS service catalog

One card per AWS service CC uses: what it does in CC, what it talks to, who may do what, encryption, data held, limits, failure behaviour and logging. Each card links to the pages that use it.

AWS Transfer Family

Purpose in ClaimClarity	The SFTP endpoint where customer labs deliver their daily 837P, 835 and 277 files
Talks to	Customer SFTP clients (SSH); Amazon S3 (writes into the customer's `landing/` prefix)
Permissions	One user per customer, signed in with the customer's SSH key. The user's home folder maps to `<customer>/landing/` only. A copy step, as the customer's SFTP copy role, then writes each file into `raw/` create-only, with Object Lock and the customer's lake key, and writes `_SUCCESS` last.
Encryption	SSH in transit; SSE-KMS with the customer's lake key at rest. The bucket policy refuses a raw write that is not create-only with Object Lock COMPLIANCE retention of at least 2,555 days.
Data held / PHI	Passes full PHI through; stores nothing itself
Scaling and limits	Managed service; one server for all customers, one user per customer
Failure behaviour	A day without the lab's `_SUCCESS` is never copied or parsed. A name that already exists in raw is refused; the lab sends a corrected file under a new name.
Logging and audit	Session logs (user, connect, file names) in CloudWatch Logs; CloudTrail records each S3 write and KMS call
Used on pages	SFTP intake · Ingestion & lake

Amazon S3

Purpose in ClaimClarity	The ClaimClarity bucket holds all customer data: each customer's lake (landing, raw, silver, gold, staging, quarantine) and chat state, one prefix per customer. The web app and the access logs have their own buckets.
Talks to	AWS Transfer Family (writes landing), the copy step (writes raw), EventBridge (Object Created events), the parser task (reads raw, writes the derived zones), the run workers (read gold), KMS (SSE-KMS on every object)
Permissions	Each customer's SFTP copy role moves files from `landing/` into `raw/` and cannot change raw. Its parser role reads raw and reads and writes `staging/`, `quarantine/`, `parquet/`, `semantic_layer/`. Its reader role reads `semantic_layer/` only. The bucket policy (one document for all customers) denies: non-TLS requests; principals without a `customer` tag; object access outside `${aws:PrincipalTag/customer}/`; listing outside that prefix; writes with any key but the principal's `kms_key_arn`; raw writes that are not create-only, COMPLIANCE-locked for 2,555 days; any raw delete. Only the Terraform deploy role may change bucket policy, lifecycle, versioning, Object Lock or encryption.
Encryption	SSE-KMS with the customer's lake key on every lake object (transcripts and the query audit use their own keys); the bucket default (platform) key is refused for customer prefixes. TLS only.
Data held / PHI	Raw, silver and quarantine: full PHI. Gold: no direct patient identifiers.
Scaling and limits	Versioning and Object Lock on. Lifecycle: noncurrent versions expire after 1 day (`semantic_layer/`, `staging/`) or 7 days (`parquet/`); none for `raw/`. The bucket policy stays the same size for any number of customers.
Failure behaviour	Conditional writes (`If-None-Match`, `If-Match`) back the batch lease and the `CURRENT` compare-and-set; a lost race fails cleanly and nothing half-written becomes visible.
Logging and audit	CloudTrail S3 data events for every object call, allowed or denied; KMS calls logged separately
Used on pages	Ingestion & lake · S3 layout · SFTP intake · Parser task · Gold publish

Amazon EventBridge

Purpose in ClaimClarity	Starts one parser task when a customer's `_SUCCESS` marker lands in raw
Talks to	Amazon S3 (Object Created events from the ClaimClarity bucket); Amazon ECS (RunTask); Amazon SQS (dead-letter queue)

Permissions	One rule per customer, matching only `<customer>/raw/received_date=*/_SUCCESS` in the ClaimClarity bucket (gate 1). Its target role, the launcher, may call `ecs:RunTask` on the parser task definitions in the ClaimClarity cluster and `iam:PassRole` on the parser roles to ECS tasks only (gate 2). It has no S3 or KMS access.
Encryption	Events carry object keys, not file contents
Data held / PHI	None. Object keys hold the customer prefix and received date, never patient data.
Scaling and limits	Delivery is at least once, so a duplicate event can start a second task; the parser's batch lease and skip of loaded files make that harmless
Failure behaviour	Retries a failed RunTask; after 3 attempts the event goes to the SQS dead-letter queue, from which a day can be redriven
Logging and audit	CloudTrail records RunTask, PassRole and failed invocations; the event id is stored with the batch in `edi.ingest_run`
Used on pages	Parse trigger · Ingestion & lake · SFTP intake

Amazon ECS on AWS Fargate

Purpose in ClaimClarity	Runs all ClaimClarity compute as three task types, with no EC2 hosts: the **chat API service**, the **run worker service** and the **parser task**.
Talks to	**Chat API:** the ALB (inbound), Cognito signing keys (through the NAT gateway and Network Firewall, Cognito hosts only), the run queue (send), the ClaimClarity bucket (conversations, run-event chunks, cancel flags, limits object), KMS. **Run worker:** the run queue (receive, delete), STS (AssumeRoleWithWebIdentity with the user's token), the ClaimClarity bucket (gold reads, chunks, answers, agent audit), KMS, Amazon Bedrock. Its egress to Bedrock goes through a VPC endpoint. **Parser task:** started by the launcher for one batch. It reads raw, writes lossless and gold under one customer's prefix, and publishes the views (Parser task). All three use VPC endpoints for AWS APIs.
Permissions	**Task execution role** (all three): pull its own image from ECR, write its own log group. **Chat API task role:** send to the run queue. It holds no customer data permissions of its own: S3 and KMS calls use the signed-in user's session credentials: Customer B's transcript role, tagged with the user. **Run worker task role:** receive and delete on the run queue, sts:AssumeRoleWithWebIdentity, Bedrock InvokeModel for Claude Haiku only. Data access uses the customer's roles and keys from the token: Customer B's reader role for gold (gates 5 and 6), the transcript role for chat state, and the audit writer role for audit records. **Parser task role:** Customer B's parser role, passed by the launcher (gate 2).

Encryption	Ephemeral task storage encrypted by Fargate. All traffic is TLS. Data at rest is in S3 under SSE-KMS with the customer's keys (lake key for gold, transcript key for chat state). Message text is also sealed with the per-user data key.
Data held / PHI	Chat API: PHI in memory only while a request runs, plus temporary chats (1 hour idle, at most 20 per user). Run worker: PHI in memory for one run. The DuckDB session has no temp directory, so nothing is spilled to disk. Parser: one batch in memory and in task storage, gone when the task ends.
Scaling and limits	**Chat API:** at least 2 tasks across two Availability Zones. Scales on CPU and request count. One process per task. **Run worker:** scales on queue depth per task (target 1 message per task). One run per worker at a time. Run deadline 110 s, inside the 120 s stop timeout. Reader credentials are renewed with the same ID token when fewer than 3 minutes remain. **Parser:** one task per batch, sized per batch type. Rolling deploys keep at least 100 % healthy, with the circuit breaker and rollback on.
Failure behaviour	**Chat API:** an unhealthy task leaves the target group, and clients reconnect to another task and replay from S3. **Run worker:** a run is not retried (the queue's maximum receive count is 1, then the dead-letter queue). Its stream stops, the client reloads, and the user retries. **Parser:** a failed batch is quarantined and alarmed.
Logging and audit	Each service logs to its own CloudWatch log group through the redaction filter, with no PHI. Audit events carry ids, counts and timings. The run worker writes one agent-audit record per SQL call under the customer's prefix. ECS API calls are recorded in CloudTrail.
Used on pages	Chat backend, Edge, Chat turn, Limits, Deploy, Monitoring

Amazon ECR

Purpose in ClaimClarity	Stores the container images for the chat API, the run worker and the parser
Talks to	CI (push, through the deploy role). ECS task execution roles (pull). Accessed through VPC endpoints from the private subnets.
Permissions	Only the deploy role may push, and only to these three repositories. Each task execution role may pull only its own repository. Nobody may delete a tagged image except the lifecycle policy.
Encryption	KMS encryption at rest. TLS in transit.
Data held / PHI	Code and dependencies only. No PHI, no secrets, no customer data, and no model credentials in any image.

Scaling and limits	Immutable tags (the commit SHA). Task definitions pin the image digest. Scan on push, and a critical finding stops the deploy. The lifecycle policy keeps the last 50 images per repository, enough to roll back.
Failure behaviour	If ECR cannot be reached, new tasks cannot start, but running tasks keep running. The ECS circuit breaker rolls back a deploy whose tasks cannot pull.
Logging and audit	Push and pull calls are in CloudTrail. Scan findings are reported to the security team.
Used on pages	Operations, Deploy

Amazon SQS (run queue)

Purpose in ClaimClarity	Decouples accepting a question from running it. The chat API queues one message per admitted run, and a free run worker takes it.
Talks to	Chat API (SendMessage). Run workers (ReceiveMessage with 20 s long polling, DeleteMessage). A dead-letter queue. CloudWatch (metrics).
Permissions	The queue policy allows SendMessage only from the chat API task role, and Receive, Delete and ChangeMessageVisibility only from the run worker task role. Access goes through a VPC endpoint, and the policy denies requests from outside it.
Encryption	SSE-KMS with a key for the queue. TLS in transit.
Data held / PHI	Ids only (message, conversation, customer, user hash) plus the user's Cognito id token, which the worker needs for the STS exchange. Never the question text, which the worker reads from the conversation object. The token is a credential: never logged, and it expires on its own.
Scaling and limits	Standard queue. Visibility timeout 150 s (above the 110 s run deadline). Message retention 5 minutes. A worker ends a run as an error without running it if it waited more than 2 minutes. Maximum receive count 1, then the dead-letter queue. Admission per customer happens before SendMessage, so the queue cannot be flooded by one customer.
Failure behaviour	A worker that dies leaves its message invisible until the timeout. It then moves to the dead-letter queue and is not run again, because a rerun would repeat model calls and SQL. The user sees the stream stop, the saved conversation reloads, and Retry runs the question again.
Logging and audit	`ApproximateNumberOfMessagesVisible` and `ApproximateAgeOfOldestMessage` drive worker scaling and alarms. Any message in the dead-letter queue raises an alarm. Queue configuration changes are recorded in CloudTrail.
Used on pages	Chat backend, Chat turn, Limits, Monitoring

Amazon CloudFront

Purpose in ClaimClarity	The one public entry point. It serves the web app and passes `/api/*` to the chat API on the same origin, so there is no CORS and the session cookie can be SameSite=Strict.
Talks to	Browsers (HTTPS). The web app bucket through origin access control (`/assets/*`). A Lambda@Edge function that returns `index.html` with a fresh CSP nonce. The Application Load Balancer (`/api/*`). AWS WAF runs on the distribution.
Permissions	Origin access control may read the web app bucket only. The Lambda@Edge role may read the `index.html` template and write its own logs. The deploy role may create invalidations. Nobody else changes the distribution, except the infrastructure pipeline.
Encryption	TLS 1.2 or higher with an ACM certificate to browsers, and HTTPS to the ALB. The web app bucket uses SSE-S3.
Data held / PHI	Caches only hashed `/assets/*` (no PHI). HTML and every `/api/*` response are `no-store` and never cached. API responses, which can hold PHI, pass through without being stored.
Scaling and limits	Managed. Origin read timeout 60 s, which is above the 15 s SSE heartbeat. `/assets/*` cached for one year (immutable). Edge features that rewrite JS, CSS or HTML stay off, because Subresource Integrity would reject the files.
Failure behaviour	An unreachable origin returns 5xx. The web app retries API calls and reconnects SSE streams. A Lambda@Edge failure stops the app from loading, but does not affect the API.
Logging and audit	Standard access logs go to the access log bucket (separate from the ClaimClarity bucket). Paths carry opaque ids, so the logs are treated as PHI-bearing: encrypted, security team only, kept 1 year. Configuration changes are recorded in CloudTrail.
Used on pages	Edge, Deploy, Chat backend

AWS WAF

Purpose in ClaimClarity	Filters every request at CloudFront before it reaches the web app or the chat API
Talks to	Attached to the CloudFront distribution. Sends logs to CloudWatch Logs.
Permissions	Only the infrastructure pipeline may change the web ACL. The operations team may read its metrics and logs.
Encryption	Not applicable (inline inspection). Its log group is encrypted with the logs key.

Data held / PHI	None stored. WAF logs are set to redact the `cookie` and `authorization` headers, and they never record request bodies.
Scaling and limits	AWS managed rule groups (common rule set, known bad inputs, IP reputation). A body size rule that allows at least the API's 1 MiB limit. A rate-based rule per client address over 5 minutes, answered with a 429 custom response (`{"error":{"code":"rate_limited"}}` and `Retry-After`).
Failure behaviour	A blocked request never reaches an origin. A rate-limit block returns 429, which the web app waits out and retries twice before it shows "Too many requests".
Logging and audit	Sampled requests and full logs (with headers redacted) in CloudWatch Logs. `BlockedRequests` and `CountedRequests` metrics per rule. Web ACL changes are recorded in CloudTrail.
Used on pages	Edge, Limits

Elastic Load Balancing (Application Load Balancer)

Purpose in ClaimClarity	Spreads `/api/*` traffic from CloudFront across the chat API tasks, and takes unhealthy tasks out of service
Talks to	CloudFront (inbound, HTTPS). The chat API target group (ECS on Fargate, private subnets).
Permissions	Its security group allows inbound HTTPS only from the CloudFront origin-facing managed prefix list. A listener rule also requires the secret origin header that CloudFront adds. Its outbound traffic may reach only the chat API tasks' port.
Encryption	HTTPS listener with an ACM certificate, and HTTPS (or TLS) to the targets.
Data held / PHI	None stored. Requests and SSE responses, which can hold PHI, pass through.
Scaling and limits	Idle timeout 60 s, which is above the 15 s SSE heartbeat. Health check `GET /api/health` , which must return 200. Deregistration delay 120 s, so open streams can drain during a deploy. Stickiness is on (load-balancer cookie, 1 hour), only so a temporary chat, which lives in one task's memory, keeps its follow-ups. Saved chats do not depend on it: their run state is in S3, so any task can serve them.
Failure behaviour	A failing target is removed. If a stream is cut during a drain, the browser reconnects with `Last-Event-ID` and replays from the stored run events.
Logging and audit	Access logs go to the access log bucket (separate from the ClaimClarity bucket) (they show full paths with opaque ids: PHI-bearing, encrypted, 1 year). CloudWatch metrics: 5xx count, response time, healthy hosts.

Used on pages	Edge, Monitoring

Amazon Cognito

Purpose in ClaimClarity	User pool federated to Microsoft Entra ID (its only identity provider). Maps each user to one customer and issues the id token that names it
Talks to	Entra ID (OIDC), the chat API sign-in callback (token endpoint), STS (via the IAM OIDC provider)
Permissions	Administrators set `custom:customer` (and `custom:transcript_admin` for customer admins). The app client cannot write these attributes. A pre-authentication check refuses users with no customer. A pre-token-generation step (v2) adds the session tags `customer`, `user` (= `sub`) and, for customer admins, `transcript_admin`, all transitive
Encryption	Tokens signed with RS256. All traffic over HTTPS
Data held / PHI	User email, Entra object id, customer mapping. No PHI
Scaling and limits	Id token 60 minutes. Refresh token 30 days, kept server-side and not given to the browser. Users are created just in time on first sign-in
Failure behaviour	No customer mapping: no token. Disabled user: no new sign-ins. Global sign-out revokes refresh tokens; issued id tokens stay valid until they expire
Logging and audit	Cognito API calls in CloudTrail. The chat API never logs tokens. See Sign-in
Used on pages	Identity & access · Sign-in · Token to credentials · Network

Microsoft Entra ID (external)

Purpose in ClaimClarity	The company login. One enterprise app for ClaimClarity with "Assignment required" on: only members of the assigned groups can sign in
Talks to	Amazon Cognito (OIDC identity provider for the user pool), the user's browser
Permissions	IT assigns groups to the app. A separate group marks customer administrators. MFA and Conditional Access follow company policy

Encryption	HTTPS. Tokens signed by the tenant's keys and verified by Cognito (issuer, tenant, app client id)
Data held / PHI	Company accounts and group membership. No PHI
Scaling and limits	Entra id token 60 minutes. The Entra browser session makes re-sign-in after the Cognito token expires a single redirect
Failure behaviour	Unassigned user: AADSTS50105. Disabled account: AADSTS50057. Unknown account: AADSTS50034. No ClaimClarity token is issued in any of these cases
Logging and audit	Entra sign-in log (successes and refusals), kept per company policy
Used on pages	Identity & access · Sign-in · Customer isolation

AWS STS

Purpose in ClaimClarity	Exchanges a user's Cognito id token for 15-minute credentials of that customer's reader role (AssumeRoleWithWebIdentity, gate 5). Also issues the AssumeRole sessions for the audit writer, the SFTP copy and support break-glass
Talks to	IAM (the OIDC provider for the Cognito user pool, role trust policies), the run workers through the STS interface endpoint
Permissions	A reader role's trust policy allows `sts:AssumeRoleWithWebIdentity` and `sts:TagSession` only for the ClaimClarity Cognito OIDC provider, only when `aud` is the ClaimClarity app client and `aws:RequestTag/customer` equals that role's customer. The audit writer trusts only the chat API role, with matching `customer` and a `user` tag. Break-glass trusts only the support role, with an external id, a `TICKET-*` reason tag, a customer tag and an operator tag
Encryption	HTTPS through the VPC interface endpoint
Data held / PHI	None. Session tags carry the customer and the Cognito `sub`
Scaling and limits	One call per run, plus one renewal per 12 minutes of a long run. Requested duration 900 s. Role maximum session duration is the AWS minimum, 3600 s. Role chaining is capped at 1 hour by AWS
Failure behaviour	`AccessDenied`, `InvalidIdentityToken` or `ExpiredTokenException` end the run with a sign-in-again message. Throttling is retried 3 times with backoff. "Revoke active sessions" on a role denies every session issued before that moment

Logging and audit	Every AssumeRole and AssumeRoleWithWebIdentity, allowed or denied, is in CloudTrail with the role and the session name `u-<sub prefix>-<request id>`
Used on pages	Identity & access · Token to credentials · Customer isolation · Network

AWS IAM

Purpose in ClaimClarity	Defines every role and what it may do. Each customer has its own lake roles (parser, reader, SFTP copy), transcript roles (user, customer admin, eval) and audit roles (writer, reader), each tagged `customer = <customer>` and `kms_key_arn = <that role's key>`. Platform roles: chat API, run workers, deploy (Terraform), key admin, support and support break-glass, eval team, and the EventBridge launcher.
Talks to	STS (role trust policies), S3 (identity policies plus the ABAC bucket policy), KMS (key policies name the roles), ECS (task roles, `iam:PassRole` for the launcher, gate 2)
Permissions	Reader: `ListBucket` (prefix `<customer>/semantic_layer/*`) and `GetObject` on `<customer>/semantic_layer/*` only, with no `GetObjectVersion`. Parser: reads raw and the derived layers, writes staging, silver, gold and quarantine of its customer. SFTP copy: `PutObject` and `PutObjectRetention` on `<customer>/raw/*`. Audit writer: `PutObject` only on `<customer>/audit/queries/*`. Chat API role: assumes the audit writer roles only, and cannot read PHI. Launcher: `ecs:RunTask` for the parser task definitions on the ClaimClarity cluster, and `iam:PassRole` for the parser roles to `ecs-tasks` only. Only the deploy role changes IAM.
Encryption	n/a. Roles carry the `kms_key_arn` tag that the bucket policy compares against every PUT
Data held / PHI	None
Scaling and limits	Roles grow linearly with customers (about 8 per customer). The bucket policy is the same size for any number of customers (ABAC on the `customer` tag). The query-audit key policy grows by 2 statements per customer, within the 32 KB key policy limit for several hundred customers
Failure behaviour	Any missing permission is an explicit `AccessDenied`, never a silent fallback. Policies that name roles that do not exist are rejected when written
Logging and audit	Every IAM change is a CloudTrail management event. Role changes outside the deploy pipeline raise an alarm
Used on pages	Identity & access · Token to credentials · Customer isolation · Encryption & keys

AWS KMS

Purpose in ClaimClarity	Customer managed keys: one lake key per customer, one transcript key per customer, one query-audit key for the platform, and a platform default key that is never valid for customer objects. Disabling a customer's key makes all of its objects unreadable at once
Talks to	S3 (SSE-KMS through `kms:ViaService`), the chat API (per-user transcript data keys), SQS, CloudWatch Logs and Secrets Manager (their own encryption)
Permissions	Key policies are authoritative and do not delegate to the account, so IAM policies cannot widen them. Lake key: `Decrypt` and `GenerateDataKey` for the customer's parser, reader and SFTP copy roles only, via S3 only, for objects under `<customer>/` only. Transcript key: user, customer admin, eval and break-glass statements, each limited by prefix or encryption context. Query-audit key: one writer and one reader statement per customer. Deploy role: `PutKeyPolicy` and lifecycle, no data use. Key admin: enable, disable, rotate, schedule deletion; no decrypt, no grants, no policy changes
Encryption	AES-256 keys held in KMS. S3 Bucket Keys are off, so each object's encryption context is its own ARN
Data held / PHI	Key material only
Scaling and limits	One `GenerateDataKey` per object written and one `Decrypt` per object read. Well within KMS request quotas at ClaimClarity's volumes. Key policy size 32 KB
Failure behaviour	Disabled or pending-deletion key: every read and write under it fails. Deletion waits 7–30 days and can be cancelled. A wrong encryption context fails to decrypt
Logging and audit	Every `Decrypt` and `GenerateDataKey` is a CloudTrail event with the role and the object ARN. Policy changes, disables and deletion schedules raise alarms
Used on pages	Token to credentials · Customer isolation · Encryption & keys · Audit & logging

Amazon Bedrock

Purpose in ClaimClarity	The only model service. Runs Claude Haiku for the analyzer: the KPI gate, the reply writer on the standard KPI path, and every step of the custom analysis agent.

Talks to	Called only by the run workers (ECS on Fargate), through the Strands `BedrockModel` (Converse streaming API). Bedrock calls nothing in ClaimClarity and has no access to the ClaimClarity bucket.
Permissions	The run worker's task role may invoke the Claude Haiku model (its inference profile and the foundation model behind it), streaming and non-streaming. No other role, model or Bedrock feature (agents, knowledge bases, fine-tuning) is allowed. The chat API and the parser cannot call Bedrock.
Encryption	TLS in transit. Bedrock does not store prompts or completions; model invocation logging is off.
Data held / PHI	Prompts contain the question, earlier questions of the conversation and query results (up to 20 rows per query, identifier columns redacted), so they are PHI. Nothing is stored after the call except the prompt cache, which keeps the shared prompt prefixes in memory for 5 minutes after their last use. Inputs and outputs are not used to train models and are not shared with the model provider. Bedrock is a HIPAA-eligible service under the AWS BAA.
Scaling and limits	On-demand, within the account's Bedrock quotas (requests and tokens per minute for the model). Per call: 4,096 output tokens, effort low (no extended thinking). Per turn: 2 calls on the standard KPI path; 1 + one per step on the custom analysis path, at most 30 agent steps.
Failure behaviour	A failed or timed-out KPI gate call sends the turn to the custom analysis path. A failed reply-writer call does the same. A failed agent call ends the turn with a retryable error. The user's Stop aborts the in-flight stream.
Logging and audit	CloudTrail records the Bedrock API calls (caller role, model, time), without prompt content. Each run records token usage per model (input, output, cache read, cache write), model time and stop reason.
Used on pages	Agent analyzer · KPI gate · Standard KPI path · Custom analysis agent · Model calls

Amazon VPC (including VPC endpoints)

Purpose in ClaimClarity	Private network for the internal ALB and all ECS tasks. Two Availability Zones. No task has a public IP or a route to the internet gateway
Talks to	CloudFront (VPC origin to the internal ALB), AWS services through endpoints, Cognito through the single filtered egress
Permissions	Security groups per tier: ALB (443 from CloudFront's origin-facing prefix list), chat API (from the ALB only), run workers and parser (no inbound), endpoints (443 from the three task groups). Endpoint policies limit each endpoint to ClaimClarity resources
Encryption	All traffic is TLS. Endpoints keep AWS traffic on the AWS network
Data held / PHI	None at rest. PHI passes between tasks and S3 through the gateway endpoint

Scaling and limits	Endpoints: S3 (gateway); STS, KMS, Bedrock runtime, ECR api, ECR dkr, CloudWatch Logs, SQS, Secrets Manager (interface, private DNS on, one network interface per AZ). Egress: one NAT gateway per AZ behind AWS Network Firewall
Failure behaviour	An endpoint is per AZ, so one AZ failing leaves the other working. If the egress path fails, signed-in users continue on the cached Cognito keys; new sign-ins fail
Logging and audit	VPC flow logs for every subnet, Network Firewall alert and flow logs, in CloudWatch Logs for 1 year
Used on pages	Network & security · Network

AWS Secrets Manager

Purpose in ClaimClarity	Holds the Cognito app client secret that the chat API uses for the code exchange at sign-in. There are no AWS access keys to store: every task uses its task role
Talks to	The chat API, through the Secrets Manager interface endpoint
Permissions	Only the chat API task role may `GetSecretValue`, on the ClaimClarity secrets only. Only the deploy role writes or rotates them
Encryption	A ClaimClarity KMS key
Data held / PHI	Secrets only. No PHI
Scaling and limits	Read at task start and cached in memory. Rotation is done by the deploy pipeline together with the Cognito app client
Failure behaviour	If the secret cannot be read at start, the chat API task fails its health check and does not take traffic
Logging and audit	Every `GetSecretValue` is a CloudTrail event. Secret values are never logged
Used on pages	Sign-in · Network

Amazon CloudWatch

Purpose in ClaimClarity	Logs, metrics and alarms for every service: chat API, run workers, run queue, parser, edge, Bedrock
Talks to	ECS tasks (logs and embedded metrics). SQS, ALB, ECS, WAF and Bedrock (their own metrics). An SNS topic for alarm notifications to on-call.

Permissions	Each task role may write only its own log group and put metrics only in the `ClaimClarity` namespace. The operations team may read logs, metrics and dashboards. No customer role has any CloudWatch access.
Encryption	Log groups are encrypted with a KMS key for logs, separate from every customer key.
Data held / PHI	No PHI. Application logs pass a redaction filter. The audit log is an allowlist of ids, counts and timings. Access logs record route shapes and truncated client addresses. Question text, answer text, SQL and row values are never logged.
Scaling and limits	Log retention: 1 year for service logs, 90 days for the edge function. Metrics carry a `customer` dimension (an id, never a name), so per-customer alarms and cost views are possible.
Failure behaviour	Logging is asynchronous. A logging outage never blocks an answer. Alarms treat missing data as breaching for the queue age and healthy-host alarms.
Logging and audit	Alarm and dashboard changes are in CloudTrail. Alarm thresholds are listed on Monitoring.
Used on pages	Operations, Monitoring, Limits

AWS CloudTrail

Purpose in ClaimClarity	The record of every AWS call: who (role and session name), what (action, object, key), and whether it was allowed. The AWS half of the audit; the agent audit is the SQL half
Talks to	The trail log bucket, CloudWatch Logs (alarms)
Permissions	Only the deploy role changes the trail. Only the security team's audit role reads the trail log bucket
Encryption	Trail logs encrypted with a dedicated KMS key. Log file validation on
Data held / PHI	Object keys, role and session names, request parameters. Object keys contain dates and file names, not patient data. No object contents
Scaling and limits	Management events plus S3 data events (read and write) for the ClaimClarity bucket. Data events are billed per event
Failure behaviour	Delivery is retried by AWS. A stopped trail or a deleted trail raises an alarm
Logging and audit	Trail log bucket with Object Lock COMPLIANCE, 6 years. Digest files allow integrity checks
Used on pages	Token to credentials · Customer isolation · Audit & logging

IAM and KMS matrix

Every role in ClaimClarity: which prefixes of the ClaimClarity bucket it may read or write, which key it may use, and who may assume it. `<c>` is one customer; each customer has its own copy of the per-customer roles and keys.

Two rules hold for every row:

- The bucket policy denies object access outside `${aws:PrincipalTag/customer}/`.
- Key policies are authoritative and allow use only through S3, for objects under the customer's own prefix.

Role	May read	May write	Key it may use	Who may assume it (trust)
Per customer				
Parser	List `<c>/`; `raw/`, `staging/`, `parquet/`, `semantic_layer/`	`staging/`, `parquet/`, `semantic_layer/`, `quarantine/` (put and delete)	`<c>`'s lake key: Decrypt, GenerateDataKey	ECS tasks in the ClaimClarity account, passed only by the launcher role
Reader	List and get `semantic_layer/` (current versions only)	nothing	`<c>`'s lake key: Decrypt, GenerateDataKey (used to read gold)	Cognito ID token for the ClaimClarity app client with `customer = <c>` (AssumeRoleWithWebIdentity)
SFTP copy	nothing	`raw/` (create only, with retention)	`<c>`'s lake key: Decrypt, GenerateDataKey (used to write raw)	The SFTP intake copy step
Transcript user	Own folder `transcripts/users/<sub>/`	Own folder (put and delete)	`<c>`'s transcript key: own folder and own data key	Cognito ID token with `customer = <c>`
Transcript admin	All of `transcripts/`	Delete in `transcripts/users/`; put and delete in `transcripts/exports/` and `transcripts/admin/`	`<c>`'s transcript key: users, exports, admin; any user data key of `<c>`	Cognito ID token with `customer = <c>` and `transcript_admin = true`

Role	May read	May write	Key it may use	Who may assume it (trust)
Eval	List and get `transcripts/exports/`	nothing	`<c>`'s transcript key: Decrypt, exports only	The eval team role
Audit writer	nothing	`audit/queries/` (create only, with retention)	Query-audit key: `<c>`'s statement	The chat API role, with session tags `customer = <c>` and `user`
Audit reader	List and get `audit/queries/`	nothing	Query-audit key: Decrypt, `<c>`'s statement	Cognito ID token with `customer = <c>` and `transcript_admin = true`
Platform				
Chat API	nothing	nothing	none	ECS tasks (chat API service). It may assume only the audit writer roles
Run worker task role	nothing in the bucket	nothing	none	ECS tasks (run worker service). It reads data only through a customer's reader role
Launcher (EventBridge target)	nothing	nothing	none	EventBridge in the ClaimClarity account. May run the parser task definitions and pass the parser roles to ECS tasks
Support	nothing	nothing	none	Support staff through company SSO. May assume only the break-glass role
Support break-glass	`<c>/transcripts/users/` for the customer named in its session tag	nothing	`<c>`'s transcript key: Decrypt only	The support role, with the external id, a `TICKET-*` reason tag and customer and operator tags
Eval team	nothing	nothing	none	Eval staff through company SSO. May assume the per-customer eval roles

Role	May read	May write	Key it may use	Who may assume it (trust)
Key admin	nothing	nothing	All keys: enable, disable, rotate, schedule deletion. No decrypt, no grants, no policy changes	Key administrators through company SSO
Deploy (Terraform)	nothing	Bucket configuration only (policy, lifecycle, versioning, Object Lock, encryption)	All keys: key policy and lifecycle. No data use	The deploy pipeline

All prefixes are under `<c>/` in the ClaimClarity bucket. Every per-customer role carries the tags `customer = <c>` and `kms_key_arn`. The bucket policy denies a PUT that names any key other than the one in the writer's `kms_key_arn` tag.

Related

See also: Identity & access · Customer isolation · Encryption keys · S3 layout

Glossary

Term	Meaning
837P	The X12 professional claim a lab sends to a payer
835	The X12 remittance advice: a payer's payment or denial for a claim
277	The X12 claim status response
CARC	Claim adjustment reason code on an 835 (e.g. CO-16: missing information). Group CO is a contractual obligation, PR is patient responsibility.
Payer / payer class	The insurer. Payer class groups payers: commercial, Medicare, Medicare Advantage, Medicaid, Medicaid MCO, Blues, other.
Current adjudication	A payer's latest decision on a claim, after reversals and corrections. Most KPIs count only these.
Reversal	An 835 with claim status 22, which takes back an earlier payment
Raw (bronze)	The files exactly as received. Written once, never changed (Object Lock).
Lossless (silver)	Parsed tables that keep every segment and can point back to the byte offsets in the raw file
Semantic (gold), `sem.*`	Curated, business-ready views (claims, remit claims, remit lines, adjustments). The chat reads these.
Escalation	The analyzer reading the lossless tables. Allowed only with a stated reason, which is recorded.
Version, CURRENT pointer	Each parser run publishes a new gold version. CURRENT names the version readers use. It is updated with compare-and-set, and old versions stay readable for sessions already open.
Batch lease	A per-customer lock held by one parser task at a time, fenced, so two tasks never write the same customer's lake
KPI gate	The first model call of a turn. It decides whether a standard KPI query answers the question exactly.
Standard KPI query	One of the fixed, reviewed SQL templates. The gate fills in its values (payer, month, CPT, CARC, payer class).

Term	Meaning
Custom analysis	The analyzer path that writes its own read-only SQL over gold, behind the SQL guard
SQL guard	The check every analyzer query passes: read-only, banned functions, join-cost limit, 60 s total query time per question
Reader session	The per-request DuckDB connection opened with the customer's temporary role credentials. It can read only that customer's current gold version.
Run	One question being answered. It has an id, a budget and a stream of numbered events.
Run events	The steps, SQL, rows and answer of a run, stored in numbered chunks so a reconnecting browser can replay them
SSE	Server-sent events: how the browser receives a run's events as they happen
Prompt caching	Bedrock reuses the unchanged start of a prompt (instructions, schema, tools) across calls, which cuts cost and latency
Gates 1–6	The AWS checks on the write path (1 EventBridge rule, 2 PassRole, 3 IAM + KMS) and the read path (4 token, 5 trust policy, 6 IAM + KMS)
PHI	Protected health information. Everything under a customer's prefix is treated as PHI.

Open decisions

The pages are written with the default shown here. Each can be changed in one place.

#	Question	Default used in these pages	Why it is open
1	SFTP: AWS Transfer Family or an SFTP host on EC2?	Transfer Family	The reference infrastructure has IT's SFTP host on EC2 with a copy script.
2	Raw-zone write rule vs Transfer Family	Transfer Family lands files in a landing prefix. A small copy step writes them into `raw/` as create-only, with Object Lock retention and the customer's KMS key.	The bucket policy demands those three on every raw write, which Transfer Family uploads don't set.
3	Does the agent audit record hold the SQL text?	Yes, the first 500 characters, stored with the customer's run data under the customer's audit key. Server logs never hold SQL.	The API contract says ids, counts and timings only; the code records SQL.
4	EventBridge retry policy for the parser launch	3 attempts, then the dead-letter queue	The AWS default retries for 24 hours.
5	Parser task credentials	Task-role credentials can outlive the task (up to about 6 hours). The boundary is the role's permissions, not the container's lifetime.	ECS behaviour. Documented as it is.
6	Reader credentials in long chats	Renewed with the same ID token when fewer than 3 minutes remain	Not yet in the code.
7	Internet egress for the chat API	NAT gateway + Network Firewall, allowing only the Cognito hosts (token keys, sign-in callback)	Alternative: no NAT, ship the token keys with each deploy.
8	Log retention	Network and app logs 1 year, CloudTrail 6 years. From the code: raw 7 years, audit 6 years, transcripts 90 days.	The 1-year and 6-year values are proposals.
9	Run limits	Run deadline 110 s. Per-customer concurrency and token budgets are settings, with values to be chosen.	Not in the code.
10	Edge	A Lambda@Edge function sets a fresh security-policy nonce on every page	Static hosting alone can't do it.

#	Question	Default used in these pages	Why it is open
11	Where the landing → raw copy step runs	A Lambda function triggered by the lab's `_SUCCESS` in `landing/`, running as the customer's SFTP copy role	Neither the code nor the design names its compute.
12	Cookie and header names (`cc_session`, `x-cc-csrf`)	Kept as they are in the code, because IT configures WAF and log redaction with them	They still carry the old "CC" name.
13	Runs on SQS and run workers	These pages describe the SQS queue and Fargate run workers. They also describe the cancel flag, limits object, CSV objects and event chunks that design needs.	Today the chat API runs each question in its own process.